

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent

12386343

Kind Code

B2

Date of Patent

August 12, 2025

Inventor(s)

Nixon; Mark J. et al.

Industrial control system architecture for real-time simulation and process control

Abstract

A Multi-Purpose Dynamic Simulation and run-time Control platform includes a virtual process environment coupled to a physical process environment, where components/nodes of the virtual and physical process environments cooperate to dynamically perform run-time process control of an industrial process plant and/or simulations thereof. Virtual components may include virtual run-time nodes and/or simulated nodes. The MPDSC includes an I/O Switch which delivers I/O data between virtual and/or physical nodes, e.g., by using publish/subscribe mechanisms, thereby virtualizing physical I/O process data delivery. Nodes serviced by the I/O Switch may include respective component behavior modules that are unaware as to whether or not they are being utilized on a virtual or physical node. Simulations may be performed in real-time and even in conjunction with run-time operations of the plant, and/or simulations may be manipulated as desired (speed, values, administration, etc.). The platform simultaneously supports simulation and run-time operations and interactions/intersections therebetween.

Inventors: Nixon; Mark J. (Thorndale, TX), Amaro, Jr.; Anthony (Cedar Park, TX), Bell; Noel Howard (Austin, TX), Caldwell; John M. (Austin, TX), Law; Gary K. (Georgetown, TX)

Applicant: FISHER-ROSEMOUNT SYSTEMS, INC. (Round Rock, TX)

Family ID: 71616019

Assignee: FISHER-ROSEMOUNT SYSTEMS, INC. (Round Rock, TX)

Appl. No.: 18/219262

Filed: July 07, 2023

Prior Publication Data

Document Identifier

US 20230350397 A1

Publication Date

Nov. 02, 2023

Related U.S. Application Data

continuation parent-doc US 17491210 20210930 US 11726463 child-doc US 18219262
continuation parent-doc US 15931998 20200514 US 11249464 20220215 child-doc US 17491210
us-provisional-application US 62859508 20190610

Publication Classification

Int. Cl.: **G05B19/418** (20060101); **G06F9/30** (20180101); **G06F13/40** (20060101); **H04L67/12** (20220101); G05B17/00 (20060101)

U.S. Cl.:

CPC **G05B19/41885** (20130101); **G05B19/4183** (20130101); **G05B19/41835** (20130101); **G05B19/41845** (20130101); **G05B19/4185** (20130101); **G05B19/41865** (20130101); **G06F9/3017** (20130101); **G06F13/4022** (20130101); **H04L67/12** (20130101); G05B17/00 (20130101); G05B2219/13125 (20130101); G05B2219/13185 (20130101); G05B2219/2214 (20130101); G05B2219/31231 (20130101); G05B2219/32301 (20130101); G05B2219/32343 (20130101); G05B2219/32355 (20130101); G05B2219/32359 (20130101); G05B2219/32407 (20130101); G05B2219/40311 (20130101)

Field of Classification Search

CPC: G06F (13/4022); G06F (9/3017); H04L (67/12); G05B (19/4183); G05B (19/41835); G05B (19/41845); G05B (19/4185); G05B (19/41865); G05B (19/41885); G05B (2219/13125); G05B (2219/13185); G05B (2219/2214); G05B (2219/31231); G05B (2219/32301); G05B (2219/32359); G05B (2219/32407); G05B (2219/33149); G05B (17/02); G05B (2219/40269); G05B (2219/40499); Y02P (90/02); Y02P (90/80)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
6298377	12/2000	Hartikainen et al.	N/A	N/A
7564805	12/2008	Cortez et al.	N/A	N/A
7684875	12/2009	Jundt et al.	N/A	N/A
8139474	12/2011	Nosley	N/A	N/A
8332567	12/2011	Burr et al.	N/A	N/A
8756041	12/2013	Maturana et al.	N/A	N/A
8762618	12/2013	Burr et al.	N/A	N/A
8977851	12/2014	Neitzel et al.	N/A	N/A
9083548	12/2014	Holmes et al.	N/A	N/A
9256222	12/2015	Blount et al.	N/A	N/A
9304511	12/2015	Blount et al.	N/A	N/A
9411769	12/2015	Erni et al.	N/A	N/A
9495313	12/2015	Burr et al.	N/A	N/A
9709978	12/2016	Asenjo et al.	N/A	N/A

9912737	12/2017	Patin et al.	N/A	N/A
9946240	12/2017	Erni	N/A	N/A
9971914	12/2017	Maturana et al.	N/A	N/A
9989958	12/2017	Asenjo et al.	N/A	N/A
10037443	12/2017	Maturana et al.	N/A	N/A
10372473	12/2018	Blount et al.	N/A	N/A
10530748	12/2019	Rotvold et al.	N/A	N/A
10620613	12/2019	Ong et al.	N/A	N/A
11320797	12/2021	Nixon et al.	N/A	N/A
2002/0107699	12/2001	Rivera	705/26.1	G06Q 30/06
2002/0135618	12/2001	Maes	715/767	G10L 15/24
2003/0038842	12/2002	Peck	715/763	G06F 11/263
2003/0055900	12/2002	Glas	709/205	H04L 12/413
2004/0022237	12/2003	Elliott	370/356	H04L 65/1104
2004/0086086	12/2003	Butler	379/22.01	H04M 1/24
2004/0158442	12/2003	Kondo	N/A	N/A
2006/0146991	12/2005	Thompson	379/67.1	H04L 67/54
2006/0265397	12/2005	Bryan	N/A	G06N 5/02
2007/0043860	12/2006	Pabari	N/A	N/A
2007/0150079	12/2006	Blevins et al.	N/A	N/A
2007/0202471	12/2006	Petruzzello	434/30	G09B 9/08
2007/0204339	12/2006	Bou-Diab	N/A	N/A
2008/0034364	12/2007	Lam et al.	N/A	N/A
2009/0089359	12/2008	Siorek et al.	N/A	N/A
2009/0089406	12/2008	Roush et al.	N/A	N/A
2011/0054640	12/2010	Law et al.	N/A	N/A
2011/0178611	12/2010	Daraiseh et al.	N/A	N/A
2012/0232875	12/2011	Devereux	N/A	N/A
2013/0136597	12/2012	Hansen et al.	N/A	N/A
2013/0205277	12/2012	Seven et al.	N/A	N/A
2013/0245843	12/2012	Bhageria et al.	N/A	N/A
2014/0070617	12/2013	Detmers et al.	N/A	N/A
2014/0107808	12/2013	Yamamoto et al.	N/A	N/A
2014/0149029	12/2013	Sakakibara et al.	N/A	N/A
2014/0266392	12/2013	Devarajan	327/376	H03K 17/04
2014/0269764	12/2013	Borgeson et al.	N/A	N/A
2014/0355581	12/2013	Pulini et al.	N/A	N/A
2015/0052519	12/2014	Yu	718/1	G06F 21/53
2015/0120550	12/2014	Jung	705/44	G06Q 20/3224
2015/0205280	12/2014	Tsuchiya et al.	N/A	N/A
2015/0381538	12/2014	Bunte	709/206	H04L 51/214
2016/0048408	12/2015	Madhu et al.	N/A	N/A
2016/0054808	12/2015	Cho	N/A	N/A
2016/0087854	12/2015	Jayanti Venkata et al.	N/A	N/A
2016/0169956	12/2015	Kim	307/125	G01R 31/68
2016/0179993	12/2015	Maturana et al.	N/A	N/A
2016/0182285	12/2015	Ferguson et al.	N/A	N/A

2016/0182288	12/2015	Maenpaa	N/A	N/A
2016/0182323	12/2015	Ferguson et al.	N/A	N/A
2016/0182693	12/2015	Ferguson et al.	N/A	N/A
2016/0359664	12/2015	Malegaonkar et al.	N/A	N/A
2017/0017211	12/2016	Periasamy et al.	N/A	N/A
2017/0025040	12/2016	Maturana et al.	N/A	N/A
2017/0032281	12/2016	Hsu	N/A	G05B 19/41875
2017/0060574	12/2016	Malladi et al.	N/A	N/A
2017/0102694	12/2016	Enver et al.	N/A	N/A
2017/0104826	12/2016	Sait et al.	N/A	N/A
2017/0118102	12/2016	Majumder	N/A	H04L 43/20
2017/0201085	12/2016	Dunn	N/A	H02H 1/0007
2017/0269577	12/2016	De Ligt et al.	N/A	N/A
2017/0300024	12/2016	Nixon et al.	N/A	N/A
2017/0329322	12/2016	Downor	N/A	N/A
2018/0024537	12/2017	Chauvet et al.	N/A	N/A
2018/0026942	12/2017	De Ligt	N/A	N/A
2018/0112795	12/2017	Anderson	N/A	N/A
2018/0113442	12/2017	Nixon	N/A	G05B 19/4184
2018/0115457	12/2017	Bonomi et al.	N/A	N/A
2018/0115516	12/2017	Rotvold et al.	N/A	N/A
2018/0115517	12/2017	Rotvold et al.	N/A	N/A
2018/0115528	12/2017	Rotvold et al.	N/A	N/A
2018/0210428	12/2017	Jundt et al.	N/A	N/A
2018/0242312	12/2017	Gandhi et al.	N/A	N/A
2018/0259222	12/2017	Murphy	N/A	N/A
2018/0260251	12/2017	Beveridge et al.	N/A	N/A
2018/0300214	12/2017	Wang	N/A	G06F 11/30
2018/0314240	12/2017	Velagapalli et al.	N/A	N/A
2018/0321662	12/2017	Nixon	N/A	G06F 13/4072
2018/0364688	12/2017	Sait et al.	N/A	N/A
2019/0041824	12/2018	Chavez et al.	N/A	N/A
2019/0068618	12/2018	Mestha	N/A	H04L 63/1466
2019/0101882	12/2018	Strinden et al.	N/A	N/A
2019/0101899	12/2018	Enver	N/A	N/A
2019/0102068	12/2018	Naidoo et al.	N/A	N/A
2019/0102076	12/2018	Naidoo et al.	N/A	N/A
2019/0173934	12/2018	Beattie, Jr.	N/A	H04L 65/1026
2019/0220703	12/2018	Prakash et al.	N/A	N/A
2019/0289091	12/2018	Burns et al.	N/A	N/A
2019/0384393	12/2018	Cruz-Hernandez	N/A	G05B 11/36
2020/0073682	12/2019	Strinden et al.	N/A	N/A
2020/0145213	12/2019	Mohan	N/A	H04L 63/102
2020/0310614	12/2019	Choi et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
103238143	12/2012	CN	N/A
104142629	12/2013	CN	N/A
104142630	12/2013	CN	N/A
105654227	12/2015	CN	N/A
106933204	12/2016	CN	N/A
106933207	12/2016	CN	N/A
107256007	12/2016	CN	N/A
107431647	12/2016	CN	N/A
107976967	12/2017	CN	N/A
108123994	12/2017	CN	N/A
108513655	12/2017	CN	N/A
109213702	12/2018	CN	N/A
109564411	12/2018	CN	N/A
109597374	12/2018	CN	N/A
109716732	12/2018	CN	N/A
2642360	12/2012	EP	N/A
2 801 940	12/2013	EP	N/A
2 840 453	12/2014	EP	N/A
3 276 887	12/2017	EP	N/A
2 429 540	12/2006	GB	N/A
2536339	12/2015	GB	N/A
2001-209407	12/2000	JP	N/A
2003-140711	12/2002	JP	N/A
2011-505613	12/2010	JP	N/A
2013-041596	12/2012	JP	N/A
2013-61762	12/2012	JP	N/A
2019-80164	12/2018	JP	N/A
2019-515375	12/2018	JP	N/A
WO-2012/047654	12/2011	WO	N/A
WO-2017/223535	12/2016	WO	N/A
WO-2018/096641	12/2017	WO	N/A
WO-2018/202276	12/2017	WO	N/A
WO-2018/204225	12/2017	WO	N/A
WO-2019/099111	12/2018	WO	N/A

OTHER PUBLICATIONS

Notification of First Office Action for Chinese Application No. 202010525946.5, dated Jul. 22, 2023. cited by applicant

Notification of First Office Action for Chinese Application No. 202010525949.9, dated Jul. 23, 2023. cited by applicant

Amaro Jr, “Improving Bandwidth Allocation in Cloud Computing Environments via “Bandwidth as a Service” Partitioning Scheme”, Texas State University, (2016). cited by applicant

Search Report for Application No. GB2008489.3, dated Feb. 19, 2021. cited by applicant

Search Report for Application No. GB2008490.1, dated Dec. 8, 2020. cited by applicant

Search Report for Application No. GB2008491.9, dated Feb. 19, 2021. cited by applicant

Search Report for Application No. GB2008492.7, dated Feb. 19, 2021. cited by applicant

Search Report for Application No. GB2008493.5, dated Feb. 19, 2021. cited by applicant
Search Report for Application No. GB2008494.3, dated Mar. 1, 2021. cited by applicant
Zhang et al., “Hierarchical real-time networked CNC system based on the transparent model of industrial Ethernet,” Int J Adv Manul Technol, 34:161-167 (2007). cited by applicant
Examination Report for Application No. GB2008494.3, dated Feb. 9, 2023. cited by applicant
Examination Report for Application No. GB2008489.3, dated Feb. 13, 2023. cited by applicant
Combined Search and Examination Report for Application No. GB2315298.6, dated Dec. 1, 2023. cited by applicant
Combined Search and Examination Report for Application No. GB2316287.8, dated Feb. 16, 2024. cited by applicant
Notification of First Office Action for Chinese Application No. 202010525409.0, dated Jul. 22, 2023. cited by applicant
Notification of First Office Action for Chinese Application No. 202010525410.3, dated Jul. 23, 2023. cited by applicant
Notification of First Office Action for Chinese Application No. 202010525418.X, dated Jul. 22, 2023. cited by applicant
Notification of First Office Action for Chinese Application No. 202010525419.4, dated Jul. 23, 2023. cited by applicant
Office Action for Japanese Application No. 2020-097851, dated Jan. 9, 2024. cited by applicant
Office Action for Japanese Application No. 2020-097854, dated Feb. 6, 2024. cited by applicant
Office Action for Japanese Application No. 2020-097855, dated Jan. 9, 2024. cited by applicant
Search Report for Application No. GB2305512.2, dated Oct. 16, 2023. cited by applicant
Search Report for Application No. GB2305513.0, dated Oct. 16, 2023. cited by applicant
Search Report for Application No. GB2305514.8, dated Oct. 16, 2023. cited by applicant
Search Report for Application No. GB2316271.1, dated Feb. 14, 2024. cited by applicant
Search Report for Application No. GB2316285.2, dated Feb. 14, 2024. cited by applicant
Search Report for Application No. GB2316288.6, dated Feb. 12, 2024. cited by applicant
Search Report for Application No. GB2316289.4, dated Feb. 12, 2024. cited by applicant
Search Report for Application No. GB2316290.2, dated Feb. 8, 2024. cited by applicant
Combined Search and Examination Report for Application No. GB2318172.0, dated Mar. 22, 2024. cited by applicant
Combined Search and Examination Report for Application No. GB2402381.4, dated Mar. 8, 2024. cited by applicant
Notification of Second Office Action for Chinese Application No. 202010525409.0, dated Mar. 6, 2024. cited by applicant
Notification of Second Office Action for Chinese Application No. 202010525418.X, dated Mar. 6, 2024. cited by applicant
Notification of Second Office Action for Chinese Application No. 202010525949.9, dated Mar. 6, 2024. cited by applicant
Search Report for Application No. GB2318173.8, dated Mar. 22, 2024. cited by applicant

Primary Examiner: Dao; Tuan C

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation of U.S. patent application Ser. No. 17/491,210, filed on Sep. 30, 2021 and entitled “Industrial Control System Architecture for Real-Time Simulation and Control,” which claims priority to U.S. patent

application Ser. No. 15/931,998, filed on May 14, 2020 and entitled “Industrial Control System Architecture for Real-Time Simulation and Control,” which claims priority to and the benefit of the filing date of U.S. Provisional Patent Application No. 62/859,508, filed on Jun. 10, 2019 and entitled “Industrial Control System Architecture for Real-Time Simulation and Control,” the entire disclosures of which are hereby expressly incorporated by reference herein.

TECHNICAL FIELD

(1) This patent application relates generally to industrial and process control systems and, more particularly, to an industrial control system that provides simulation of process control and/or run-time actual process control using virtualized components.

BACKGROUND

(2) Process or industrial control systems, like those used in chemical, petroleum or other industrial process plants to produce physical products from materials, typically include one or more process controllers communicatively coupled to one or more field devices via analog, digital or combined analog/digital buses, or via a wireless communication link or network. The field devices, which may be, for example, valves, valve positioners, switches and transmitters (e.g., temperature, pressure, level and flow rate sensors), are located within the process environment and generally perform physical or process control functions such as opening or closing valves, measuring process parameters, etc., to control one or more processes executing within the process plant or system. Smart field devices, such as the field devices conforming to the well-known FOUNDATION® Fieldbus protocol may also perform control calculations, alarming functions, and other control functions commonly implemented within the controller. The process controllers, which may be centrally located but which may also be located within the plant environment in a distributed manner, receive signals indicative of process measurements made by the field devices and/or other information pertaining to the field devices and execute a controller application that runs, for example, different control modules that make process control decisions, generate control signals based on the received information and coordinate with the control modules or blocks being performed in the field devices, such as HART®, WirelessHART®, and FOUNDATION® Fieldbus field devices. The control modules in the controller send the control signals over the communication lines or links to the field devices to thereby control the operation of at least a portion of the process plant or system.

(3) Information from the field devices and the controller is usually made available from the controllers over a data highway to one or more other hardware devices, such as operator workstations, personal computers or computing devices, data historians, report generators, centralized databases, or other centralized administrative computing devices that are typically placed in control rooms or other locations away from the harsher plant environment. Each of these hardware devices typically is centralized across the process plant or across a portion of the process plant. These hardware devices execute applications that may, for example, enable an engineer to configure portions of the process or an operator to perform functions with respect to controlling a process and/or operating the process plant, such as changing settings of the process control routine, modifying the operation of the control modules within the controllers or the field devices, viewing the current state of the process, viewing alarms generated by field devices and controllers, simulating the operation of the process for the purpose of training personnel or testing the process control software, keeping and updating a configuration database, etc. The data highway utilized by the hardware devices, controllers and field devices may include a wired communication path, a wireless communication path, or a combination of wired and wireless communication paths.

(4) As an example, the DeltaV™ control system, sold by Emerson Process Management, includes multiple applications stored within and executed by different devices located at diverse places within a process plant. A configuration application, which resides in one or more workstations or computing devices, enables users to create or change process control modules and to download

these process control modules via a data highway to dedicated distributed controllers. Typically, these control modules are made up of communicatively interconnected function blocks, which are objects in an object-oriented programming protocol that perform functions within the control scheme based on inputs thereto and that provide outputs to other function blocks within the control scheme. The configuration application may also allow a configuration designer to create or change operator interfaces that are used by a viewing application to display data to an operator and to enable the operator to change settings, such as set points, within the process control routines. Each dedicated controller and, in some cases, one or more field devices, stores and executes a respective controller application that runs the control modules assigned and downloaded thereto to implement actual process control functionality. The viewing applications, which may be executed on one or more operator workstations (or on one or more remote computing devices in communicative connection with the operator workstations and the data highway), receive data from the controller application via the data highway and display this data to process control system designers, operators, or users using the user interfaces, and may provide any of a number of different views, such as an operator's view, an engineer's view, a technician's view, etc. A data historian application is typically stored in and executed by a data historian device that collects and stores some or all of the data provided across the data highway while a configuration database application may run in a still further computer attached to the data highway to store the current process control routine configuration and data associated therewith. Alternatively, the configuration database may be located in the same workstation as the configuration application.

(5) The architecture of currently known process control plants and process control systems is strongly influenced by limited controller and device memory, communications bandwidth, and controller and device processor capability. For example, in currently known process control system architectures, the use of dynamic and static non-volatile memory in the controller is usually minimized or, at the least, managed carefully. As a result, during system configuration (e.g., a priori), a user or configuration engineer typically must choose which data in the controller is to be archived or saved, the frequency at which it will be saved, and whether or not compression is used, and the controller is accordingly configured with this limited set of data rules. Consequently, data which could be useful in troubleshooting and process analysis is often not archived, and if it is collected, the useful information may have been lost due to data compression.

(6) Additionally, to minimize controller memory usage in currently known process control systems, selected data that is to be archived or saved (as indicated by the configuration of the controller) is reported to the workstation or computing device for storage at an appropriate data historian or data silo. The current techniques used to report the data poorly utilizes communication resources and induces excessive controller loading. Additionally, due to the time delays in communication and sampling at the historian or silo, the data collection and time stamping is often out of sync with the actual process.

(7) Similarly, in batch process control systems, to minimize controller memory usage, batch recipes and snapshots of controller configuration typically remain stored at a centralized administrative computing device or location (e.g., at a data silo or historian), and are only transferred to a controller when needed. Such a strategy introduces significant burst loads in the controller and in communications between the workstation or centralized administrative computing device and the controller.

(8) Further, the current architecture of industrial control systems, such as process control systems, is largely hardware driven in that various functions, such as control functions, input/output (I/O) functions, user interface functions, etc. are performed in and are tied to specific pieces of hardware (e.g., user workstations or interface devices, process controllers, safety system controllers, dedicated I/O devices, marshalled I/O devices, field devices, safety logic solvers, etc.) and remain stored in the specific hardware at all times. For example, in current process control systems, interconnections between controllers and I/O devices (e.g., either individual I/O devices, or banks

of marshalled I/O devices) are configured based on particular hardware, and consequently, physical I/O relationships are tightly bound, most commonly in a one-to-one manner, e.g., I/O device to controller, another I/O device to another controller, etc. This architectural design limits the resiliency, reliability, availability, responsiveness, and the elasticity of the control system, as these systems are difficult to change or reconfigure quickly, are tightly tied to proprietary hardware used to implement proprietary control software, require hardware redundancy on a device by device basis that can be expensive to implement, and are not easily scalable or able to be upgraded without adding additional hardware that needs to be configured in particular manners, for example, due to size limitations of individual devices such as physical process controllers, particular characteristics and capabilities of I/O devices, etc.

(9) Some attempts at addressing these issues include virtualizing physical controllers; however, controller virtualization has been limited to off-line development and test systems, and has not been widely utilized, if at all, for run-time process control in physical plant and/or production environments. Moreover, both virtualized controllers and physical controllers remain subject to the limitations of physical I/O devices, such as performance, bandwidth, throughput, etc.

SUMMARY

(10) A novel, multi-purpose hardware/software architecture or platform for dynamic simulation and/or run-time production process control that enables an industrial or process control system of an industrial process plant to be more resilient, responsive, and elastic as compared to known industrial or process control systems. More particularly, this novel architecture or platform, to a large extent, decouples the hardware used in the control system from the software that governs the behavior of the hardware, making the system easier to scale, reconfigure, and change, as well as improving overall system reliability, availability, and performance. For ease of discussion, the novel Multi-Purpose Dynamic Simulation and run-time Control platform or architecture is referred to interchangeably herein by the “MPDSC,” “the MPDSC platform,” “the MPDSC system, or “the MPDSC architecture.”

(11) Generally speaking, the MPDSC includes a virtual process environment coupled to a physical process environment, where components of the virtual and physical environments cooperate to perform dynamic simulation and/or run-time (e.g., actual or operational) production process control of the industrial process plant. For run-time process control, the MPDSC platform supports Process Control Systems (PCSs) which may include virtual components, physical components, and/or both virtual and physical components that cooperate to monitor and control batch and/or continuous industrial processes during run-time of the plant. In some arrangements, the MPDSC platform also supports Safety Instrumented Systems (SISs) which may have both virtual and physical components that cooperate to maintain safe operations and provide failsafe operations of the plant during run-time. Virtual nodes that perform run-time, operational process control and/or related run-time functions (e.g., in conjunction with one or more physical devices or components disposed in the physical environment of the process plant) are referred to herein as “virtual run-time nodes.”

(12) The MPDSC platform may additionally or alternatively support a simulation environment or space which may be utilized for control and/or safety system and/or component testing, validation, verification, and/or check out, operator training, case studies, on-going process improvements, etc. For simulation purposes, the MPDSC platform provides virtual nodes that simulate one or more physical devices, modules, or components that are deployable within the physical process environment. Virtual nodes that simulate various devices, modules, and/or components that are deployable in the physical environment of the process plant are referred to herein as “simulated nodes.”

(13) The simulation provided by the MPDSC platform is “dynamic,” as simulations of the process plant or portions thereof may execute in real-time, thereby mirroring the timing and other behaviors that (would) occur during run-time, and/or simulations may be manipulated to execute at a faster or slower pace than real-time execution, to use different values, initial conditions, intermediate

conditions, etc. The simulation space provides various features such as, for example, save/restore capabilities for various simulated scenarios, editable initial and/or intermediate conditions for various scenarios, coordination with third-party simulators via various protocols, testing and checkout of operator display views, real-time simulation of one or more portions of the PCS, speed-up and/or slow-down of the simulation of the one or more portions of the PCS, simulations that include simulated components operating in conjunction with actual (e.g., non-simulated) virtual and/or physical components, change management and synchronization with plant configuration, etc. The MPDSC platform is able to simultaneously support both simulation and run-time operations, various interactions and intersections between simulation and run-time operations (e.g., testing of upgrades, patches, “what-if” scenarios, configuration changes, user input changes, and the like. Further, with the MPDSC platform, system redundancy, fault tolerance, switchovers, planned maintenance, etc. are able to be seamlessly and easily accomplished.

(14) One of the key components of the MPDSC architecture is referred to herein as an “I/O Switch” or an “I/O Server,” which generally abstracts I/O of the process plant away from being specifically and directly associated with particular hardware, as well as performs other functions related to I/O for simulation and/or run-time process control. The I/O Switch or I/O Server is a node that, during run-time, delivers I/O data between virtual and/or physical nodes of the process plant. Each node (whether virtual or physical) is typically a component of the process plant that is identified within the MPDSC system by a unique name, tag, or identifier. For example, a node may be a physical or virtual controller, a field device, a safety information controller, a safety logic solver, an I/O card or device (e.g., a wireless I/O device, an Ethernet I/O device, a marshalled I/O cabinet or components thereof, etc.), an operator workstation, another type of user interface device, a tool (e.g., diagnostic tool, simulation tool, etc.), a gateway, a data storage device, or other type of component. Generally speaking, the I/O Switch operates as a data broker between virtual and/or physical nodes, where the I/O Switch delivers or switches I/O data (also referred to interchangeably herein as “process I/O data,” or “PIO data”) between source nodes (whether virtual or physical) and destination nodes (whether virtual or physical). In a sense, rather than tightly and/or specifically coupling physical I/O cards to different source and destination nodes to deliver process I/O, the I/O Switch virtualizes the physical delivery mechanisms of process I/O data between nodes of the MPDSC system, and loosens the bindings between hardware components that are utilized for the purposes of delivering I/O data. Moreover, the I/O Switch is configured to be able to deliver I/O data quickly enough (e.g., with sufficiently small delay) to support run-time, actual production process control and/or real-time simulation of production process control.

(15) Generally speaking, virtual and physical nodes/components that are serviced by the I/O Switch or I/O Server include respective modules that govern the behavior of virtual and physical nodes or components. Such modules are referred to herein as “component behavior modules” or “CBMs,” examples of which include control modules in controllers, safety logic in safety logic solvers, and other types of modules that govern the behavior and operations of the components in which the modules are stored and executed, and at least in part by operating on I/O data. Within the MPDSC system, CBMs that operate on the process-related payload of the I/O data are agnostic or unaware of the I/O Switch and its role in delivering I/O data to/from the component behavior modules. That is, the CBMs are unaware of whether or not their respective I/O delivery mechanism to/from their host node is a physical I/O card or the I/O Switch.

(16) As such, the I/O Switch may be viewed as a switching fabric, router, and/or delivery mechanism of I/O data that is transparent to Component Behavior Modules of the MPDSC system. To this end, the I/O Switch delivers I/O data between virtual and/or physical nodes via respective publish/subscribe layers of the nodes (also referred to interchangeably herein as a “Pub/Sub layer”) and a virtual communication network via which I/O payload or data is transferred between nodes. For example, a sending or publishing node may include a respective Pub/Sub layer that publishes, to the virtual communication network, data generated by the component behavior module of the

sending node. The I/O Switch may deliver or switch the published data to nodes that are receivers of or subscribers to the data, and each subscriber node may recover the data via its respective Pub/Sub layer for consumption by its respective component behavior module. As such, the I/O Switch brokers data between publisher nodes and subscriber nodes on a demand basis, in accordance with the defined relationships of the nodes within the MPDSC system.

(17) As utilized herein, the “physical environment” of the MPDSC platform or system refers to the production plant or environment in which physical, tangible components (e.g., field devices, tanks, valves, actuators, heaters, evaporators, sensors, etc.) are utilized to transform, via run-time controlled processes, physical materials into physical products. Accordingly, the “physical environment” is interchangeably referred to herein as the “plant environment” of the industrial process plant. As discussed above, the physical or plant environment includes a front-end portion in which physical or hardware components of the MPDSC system such as field devices, sensors, transmitters, switches, positioners, tanks, heaters, etc. are disposed and operate on physical materials to produce physical products. As such, the “front-end” portion of the physical environment is interchangeably referred to herein as the “field” or “site” portion of the physical environment of the process plant.

(18) The physical or plant environment of the MPDSC system also includes a back-end portion in which physical or hardware components such as operator workstations, personal computers or computing devices, data historians, report generators, centralized databases, and/or other centralized (or at least partly centralized) administrative computing devices execute applications to, for example, configure the process plant and/or its components, view and monitor run-time operations of the plant, respond to alerts or alarms during run-time operations of the plant, adjust parameters during run-time operations of the plant, generate reports, store and analyze data, and the like. The back-end portion of the physical environment of the MPDSC system may be located in areas that are protected from the harsher field environment, such as in an on-site, enclosed room and/or in locations that are off-site or remote from the field environment.

(19) The virtual environment of the MPDSC system is implemented using physical or hardware computing devices that are particularly configured and interconnected to provide a platform that supports the virtualization of various physical process plant components, as is described in more detail in later sections of this disclosure. Generally speaking, the physical computing devices that provide and support the virtualization platform may be physically located on-site at the plant (e.g., in protected, enclosed areas of the field environment), may be physically located off-site, or may be physically and distributively located among various on-site and off-site locations. The physical computing devices that provide and support the virtualization platform may be interconnected via any number of physical data links and/or or communication/data networks. Generally speaking, the physical computing devices, data links, and communication/data networks form a computing platform on which various logical or virtual components of the MPDSC system reside, where the various logical or virtual components may be utilized for run-time process control in conjunction with components of the physical process environment, and/or for process simulation purposes.

(20) The logical or virtual components residing in the virtual environment of the MPDSC system may operate to provide simulation of one or more actual or planned physical portions of the MPDSC system (e.g., in real-time, at faster speeds, and/or at slower speeds, if desired). In some implementations, the logical or virtual components residing in the virtual environment of the MPDSC system and the physical components residing in the physical environment of the MPDSC system may cooperatively operate to provide simulation and/or run-time, actual production process control. As such, in some embodiments, the physical and virtual environments of the MPDSC platform are communicatively connected via one or more communication links and/or networks, where the communication links and/or networks may include any number of wired links and/or networks, wireless links and/or networks, private links and/or networks, public links and/or networks. Embodiments of stand-alone, virtual real-time simulation and embodiments of

cooperation between the logical/virtual and physical components of the industrial control system and communicative connections therebetween are described in more detail in later sections of this disclosure.

(21) Further, the virtual and physical environments of the MPDSC platform utilize or share a common (e.g., the same) system configuration database, which is referred to herein as the “MPDSC system configuration database.” As such, via the MPDSC system configuration database, various virtual and physical components may be uniquely identified within the MPDSC platform across both virtual and physical environments, and intersections between simulation and run-time operations (e.g., testing, switchovers, etc.) are able to be seamlessly and easily accomplished.

(22) In an embodiment, a system, such as a multi-purpose dynamic simulation and run-time industrial or process control (MPDSC) system, comprises a process control system that includes a field device, where the field device is disposed in a physical environment of an industrial process plant and performs a physical function. Additionally, the process control system includes an I/O switch communicatively disposed between the field device and a virtual node, where the virtual node is disposed in a virtual environment of the industrial process plant. The I/O switch is a subscriber to first data that is generated by the field device and that has been published, and the I/O switch is a publisher of second data indicative of the first data generated by the field device. The process control system further includes the virtual node, where the virtual node is a subscriber to the second data corresponding to the field device and published by the I/O switch. The virtual node includes a component behavior module that operates on the second data corresponding to the field device to thereby generate a control signal to modify a behavior of another node of the process control system. The field device, the virtual node, and the another node operate in conjunction during run-time operations of the industrial process plant to control an industrial process, for example.

(23) In an embodiment, a method of controlling a node of a process control system of an industrial process plant includes communicatively connecting a field device and a virtual node via an I/O switch disposed between the field device and the virtual node so that the field device, the I/O switch, and the virtual node operate in conjunction during run-time operations of the industrial process plant to control an industrial process. The field device is disposed in a physical environment of the industrial process plant and performs a physical function. The I/O switch is a subscriber to first data that is generated by the field device and that has been published, and the I/O switch is a publisher of second data indicative of the first data generated by the field device. The virtual node is disposed in a virtual environment of the industrial process plant, and the virtual node is a subscriber to the second data corresponding to the field device and published by the I/O switch. During the run-time operations of the industrial process plant, the method includes receiving, by the virtual node, the second data corresponding to the field device and published by the I/O switch; operating on, by utilizing a component behavior module of the virtual node, the second data, thereby generating a control signal based on the second data; and causing an indication of the control signal to be transmitted to another node of the process control system, thereby modifying a behavior of the another node.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. 1 is a block diagram of an example multi-purpose dynamic simulation and run-time industrial or process control (MPDSC) system that provides dynamic simulation and/or run-time industrial or process control of a process plant.

(2) FIG. 2 is a block diagram illustrating embodiments of example virtual node architectures which may be included in the MPDSC system of FIG. 1.

- (3) FIG. 3 is an example arrangement of a physical plant environment which the MPDSC system of FIG. 1 may support.
- (4) FIG. 4 depicts an example arrangement of multiple I/O Switches, which may be at least partially implemented in the MPDSC system of FIG. 1.
- (5) FIG. 5 is a block diagram illustrating an example virtualization management system via which at least portions of the MPDSC system of FIG. 1 may be configured and administered.
- (6) FIG. 6 is a flow diagram of an example method of controlling a node of a process control system of an industrial process plant.

DETAILED DESCRIPTION

(7) FIG. 1 is a block diagram of an example multi-purpose dynamic simulation and run-time industrial or process control (MPDSC) system or platform **10** that provides dynamic simulation and/or run-time process control of an industrial or process plant. The example MPDSC platform **10** supports dynamic simulation of process control of the industrial process plant and/or real-time process control using virtualized devices. For example, the virtualized devices may be utilized by the MPDSC platform **10** for physical production purposes during run-time of the process plant. As shown in FIG. 1, the MPDSC platform **10** includes a virtual plant environment portion **12** and a physical plant environment portion **15**; however, in embodiments in which the MPDSC platform **10** is utilized solely for real-time simulation purposes, the physical plant environment portion **15** may be omitted. Generally speaking, the virtual plant environment portion **12** and the physical plant environment portion **15** collectively form one or more Operations Technology (OT) layers **18** of the industrial process plant, which may provide generated data to one or more Information Technology (IT) layers **20** associated with the industrial process plant and/or to networks that are external to the IT layers, e.g., for enterprise and/or external use by one or more applications **21a-21n**, **22a-22m**. For example, applications **21a-21m** may be provided by the enterprise that owns/operates the MPDSC platform **10** and may execute in an IT layer associated with the enterprise, and applications **22a-22m** may be Internet of Things (IoT) and/or Industrial Internet of Things (IIoT) applications provided by respective third-parties that execute in remote networks which are accessed via the Internet or other public network.

(8) The IT layers **20** of the enterprise may be implemented in any number of locally and/or remotely disposed computing devices, such as one or more local and/or remote server banks, one or more computing clouds, etc., and may include any number of applications or consumers **21a-21n** of data generated by the OT layers **18** of the enterprise. Typically (but not necessarily), and as shown in FIG. 1, the OT layers **18** are communicatively coupled to the IT layers **20** via one or more public and/or private communications networks **24** and one or more edge gateway systems **28**, where the edge gateway system(s) **28** provide security and efficiencies of data delivery between the OT layers **18**/MPDSC platform **10** and the IT layers **20**. Further, the one or more edge gateway systems **28** may provide security and efficiencies of data delivery between the MPDSC platform **10** and external networks and/or external applications **22**.

(9) In FIG. 1, the virtual plant environment **12** of the MPDSC platform **10** includes one or more Virtual Nodes (VNs) **30a**, **30b**, **30c**, . . . , **30p** that are communicatively connected with an I/O Switch (which is interchangeably referred to herein as an "I/O Server") **25**. Generally speaking, each Virtual Node **30x** is a virtual machine that simulates or virtualizes the behavior of a respective physical component which may be operable within the physical plant environment **15**. Said another way, each Virtual Node **30x** within the virtual plant environment **12** processes data and behaves in the same manner in which its physical counterpart within the physical plant environment **15** would process data and behave.

(10) In particular, each Virtual Node **30x** includes a framework and one or more subsystems **32** that allow the VN **30x** to communicate with other Virtual Nodes **30x** within the virtual plant environment **12** and/or with one or more Physical Nodes (PNs) within the physical plant environment **15**, such as Physical Nodes **40a**, **40b**, **40c** and the Edge Gateway System **28** (where

the Edge Gateway **28** may be viewed as a particular type of PN), each of which includes respective frameworks and respective one or more subsystems **42x** that allow communications with Virtual Nodes **30x**. Additionally, the MPDSC platform **10** may include any number of other physical nodes or components (e.g., PNa-PNk).

(11) Virtual Nodes may virtualize different types of Physical Nodes. Generally speaking, as utilized herein, a “Physical Node” is a physical device or component of the MPDSC platform **10** that includes hardware and that transmits and/or receives data to/from other devices or components (whether virtual or physical). Examples of types Physical Nodes which may be represented by respective Virtual Nodes include, but are not limited to: various types of process control controllers; various types of local and/or remote I/O devices or cards (such as Wireless I/O Cards (WIOCs), Ethernet I/O Cards (EIOCs), etc.), and various components of I/O electronic marshalling systems (such as CHARacterization Modules (CHARMs), terminal blocks, CHARM I/O Cards (CIOCs), etc.); various types of Safety Instrumented System (SIS) nodes such as safety controllers, safety logic solvers, SIS repeaters, safety network bridges, safety network gateways, etc.; user interface devices including local and/or remote physical operator workstations, mobile devices, and/or other computing devices that provide user interfaces for run-time operations and/or for other purposes related to the MPDSC platform **10**; local and/or remote physical computing devices that provide tools related to the MPDSC platform **10**, such as control configuration tools, data consolidation and viewing tools, data analytics and analytics configuration tools, display view configuration tools, diagnostic tools, asset management tools, application stations, etc.; various types of gateways utilized within and/or by the MPDSC platform **10**, such as wireless gateways, safety gateways, firewalls, edge gateways, field gateways, inter-system gateways, etc.; and other types of physical nodes which may be utilized within a physical plant environment **15**.

(12) In some embodiments, a single VN **30x** may represent or virtualize an entire Physical Node. In some embodiments, a single VN **30x** may represent or virtualize one or more portions of a particular Physical Node. For example, a single VN **30x** may represent or virtualize a particular module or group of modules executing or residing at the particular PN, where such modules may include software modules or components, firmware modules components, and/or hardware modules or components. For example, a first VN may represent or virtualize an entire application executing on the particular PN (such as the entirety of a control module which is executable on a physical process controller), a second VN may represent or virtualize a subset of the entire application executing on the particular PN (such as a particular control model, function, or routine utilized by the control module), a third VN may represent or virtualize the behavior of a protocol-specific I/O card or device associated with the particular PN, and/or a fourth VN may represent or virtualize a PN sub-component as granular as a hardware sub-component of the particular PN (e.g., a port or other interface, a bus, a transceiver, a chip, a board, etc.), or a firmware or software sub-component of the particular PN (e.g., a module, a routine, a function or behavior, a networking address of the particular PN, such as a MAC (Media Access Control) address or other type of addresses, etc.).

(13) Examples of types of physical I/O cards which may be utilized within the physical plant environment **15** and which may be represented and/or virtualized by Virtual Nodes **30x** of the MPDSC **10** include, but are not limited to: discrete output cards, including high density, intrinsically safe, and redundant discrete output cards; discrete input cards, including high density, intrinsically safe, and redundant discrete input cards; analog input cards, including analog input cards that support the HART® (Highway Addressable Remote Transducer) communication protocol, redundant analog input cards that support HART, high density redundant analog input cards that support HART, and fast analog input cards; analog output cards, including analog output cards that support HART, redundant analog output cards that support HART, high density redundant analog output cards that support HART, and fast analog output cards; serial cards, including redundant, programmable, and redundant programmable serial cards; interface cards for

discrete actuators and/or sensors, RTD (Resistance Temperature Detector) cards, thermocouple cards, millivolt cards, isolated temperature cards, multifunction cards, sequence of events cards, that support the Fieldbus communication protocol, redundant Fieldbus-supporting cards, cards that support the Profibus communication protocol, redundant Profibus-supporting cards, and other types of physical cards.

(14) As illustrated in FIG. 1, the VNs **30a**, **30b**, **30c**, . . . , **30p** communicate with each other and with the I/O Switch **25** via respective Publication/Subscription Layers **32a**, **32b**, **32c**, . . . , **32p** and **35**, which are referred to interchangeably herein as “Pub/Sub Layers,” and which are denoted in FIG. 1 by the respective hash-marked portions **32x**, **35** of each VN **30x** and of the I/O Switch **25**. In embodiments of the MPDSC platform **10** in which Physical Nodes are included, such as illustrated in FIG. 1, at least some of the PNs **28**, **40a**, **40b**, **40c** disposed in the physical plant environment **15** may include respective Pub/Sub Layers **38**, **42a**, **42b**, **42c**. In a sense, the Pub/Sub Layers **32x**, **35** (and, in some arrangements, Pub/Sub Layers **38**, **42x**) serve as a virtual communication network **45** via which various Virtual Nodes **30x** and the I/O Switch **25** (and, in some arrangements, the Physical Nodes **40x**) may communicate abstracted I/O data. In an example implementation, each Pub/Sub Layer **32x**, **35**, **38**, **42x** is a respective interface to the virtual communication network **45**.

(15) The virtual communication network **45** may be implemented by one or more physical communications and/or data links and/or networks, which may include wired and/or wireless networks, and which may be implemented using any suitable technology, such as Ethernet, optical, IP, another type other packet network, etc. Data is communicated between nodes of the via the virtual communication network **45** via publication and subscription, and any or more suitable communication protocols that support publication and subscription may be utilized within the virtual communication network **45** for the delivery of data. For example, private packet protocols and/or public or standardized packet protocols (such as IPv6, IoT, and/or IIoT protocols) may be utilized for publication of and subscription to I/O data that is delivered between various nodes **30x**, **28**, **40x** of the virtual communication network **45** and, optionally, to other applications **21x**, **22x**, e.g., by way of edge gateway systems **28**.

(16) For example, each node of the virtual communication network **45** (e.g., Virtual Nodes **30x**, I/O switch **25**, Physical Nodes **28**, **40x**, etc.) may publish I/O data to the virtual communication network **45** via its respective Pub/Sub Layer (e.g., via Pub/Sub Layers **32x**, **35**, **38**, **42x**, etc.), and each node of the virtual communication network **45** (e.g., Virtual Nodes **30x**, I/O switch **25**, Physical Nodes **28**, **40x**, etc.) may subscribe to and obtain I/O data that is published to the virtual communication network **45** via its respective Pub/Sub Layer (e.g., via Pub/Sub Layers **32x**, **35**, **38**, **42x**, etc.). Typically, subscriptions to various published data have a one-to-one correspondence between the I/O Switch **25** and each of the other nodes **30x**, **28**, **40x**. In a preferred embodiment, each node **30x**, **28**, **40x** of the virtual communication network **45** accepts subscriptions only from the I/O Switch **25** and not from other nodes, and each node **30x**, **28**, **40x** subscribes to only I/O data that is published by the I/O Switch **25** (where the I/O data published by the I/O Switch **25** may be forwarded data that was generated by other nodes and to which the I/O Switch **25** has subscribed) and not to I/O data published by other nodes. As such, in this preferred one-to-one embodiment, undirected graphs of publication/subscription relationships are restricted as compared to embodiments that allow nodes to have multiple subscriptions with multiple other nodes, thereby reducing network complexity, simplifying network diagnosis and management, and decreasing network load/utilization. To further reduce network complexity, simplify network diagnosis and management, and decrease network load/utilization, additionally or alternatively each node **30x**, **28**, **40x** of the virtual communication network **45** may send, to the I/O Switch **25** via publication, only data that is to be forwarded by the I/O Switch **25** to other nodes of the MPDSC platform **10**. Of course, in some embodiments, at least portions of undirected graphs may be implemented in one-to-many, many-to-one, and/or many-to-many relationships, if desired.

(17) Generally speaking, within the MPDSC platform **10**, process-related payload data (e.g., as

generated by CBMs and/or physical components of the process plant) is abstracted and published as I/O data, and may be delivered to subscriber nodes of the virtual communication network **45** on a demand basis. That is, I/O data may be delivered (e.g., via publication) when the publishing node determines that the process-related data payload requires a new publish event. For example, a publishing node may automatically publish, to the virtual communication network, a certain type of sensor-generated data as I/O data when the sensor-generated data value changes. Optionally, the publishing node may publish, as I/O data, the sensor-generated data value periodically (e.g., every five seconds, ten seconds, one minute, etc.) even when the sensor-generated data value has not changed, e.g., to thereby mitigate lost messages and other fidelity issues which may possibly arise. As such, in some embodiments, no explicit subscription rate is required and/or utilized by subscriber nodes. Consequently, execution of the processing logic (e.g., the component behavior module) at each subscriber node is driven by incoming data that is subscribed to and received via virtual communication network **45**, and accordingly, resources utilized by each node and by the virtual communication network **45** may be more efficiently utilized.

(18) In an example illustrative scenario, VN **30a** may generate and publish, as I/O data during run-time, a certain set of process-related payload data, e.g., “Data1,” to its Pub/Sub Layer **32a**. The I/O Switch or Server **25** may have a subscription to I/O data that includes Data1, and may obtain the process-related payload Data1 via the virtual communication network **45**, its respective Pub/Sub Layer **35**, and its respective subscription. In turn, the I/O Switch or Server **25** may publish the obtained, process-related payload Data1 as I/O data to the virtual communication network **45** via its Pub/Sub Layer **35** for receipt by those nodes that have subscriptions to I/O data that includes Data1. For example, VN **30b** may have a subscription to I/O data that includes Data1, and upon publication of Data1 as I/O data via the Pub/Sub Layer **35** of the I/O Switch **25**, VN **30b** may obtain the process-related payload Data1 via the virtual communication network **45**, its respective Pub/Sub Layer **32b**, and its subscription thereto. Subsequently, VN **30b** may operate upon the obtained process-related payload Data1 values.

(19) Thus, the MPDSC platform **10** abstracts multiple different types of I/O that is utilized in and native to various physical components of industrial process plants (e.g., discrete output, discrete input, analog output, analog input, serial I/O, Railbus, HART, wireless HART, Fieldbus, Profibus, Ethernet, Advanced Physical Layer, and/or any other types of I/O) for delivery amongst virtual and physical nodes via the I/O Switch **25** and the virtual communication network **45** via publication and subscription. Each node of the virtual communication network **45** may perform respective abstraction and de-abstraction (e.g., recovery) of I/O data that it sends and receives via its respective Pub/Sub Layer and, optionally, other subsystems.

(20) Virtual Nodes

(21) To illustrate I/O abstraction and de-abstraction, FIG. 2 illustrates two example architectural embodiments **52a**, **52b** of a Virtual Node **30x**. Generally speaking, a Virtual Node **30x** simulates and/or virtualizes a Physical Node or physical component which may be utilized and operate within the physical plant environment **15**. Each VN **52a**, **52b** includes a respective Pub/Sub Layer **55a**, **55b** via which the VN **52a**, **52b** communicates with the I/O Switch **25**. Additionally, each VN **52a**, **52b** includes a respective Component Behavior Module **58a**, **58b**, which is interchangeably referred to herein as a “Component Business Logic Module” **58a**, **58b** or a “CBM” **58a**, **58b**. Generally speaking, a CBM **58x** is a module whose execution governs the operating behavior of its respective Virtual Node **30x**, e.g., at an application level. For example, the CBM **58x** of a Virtual Controller Node may be a particular instance of a control module, where other instances of the control module may be downloaded into physical controllers disposed in the physical plant environment **15** for execution during run-time of the physical plant **15**. In another example, the CBM **58x** of a Virtual CIOC Node may be a particular instance of a remote I/O module, where other instances of the remote I/O module may execute, during run-time of the physical plant environment **15**, at physical CIOCs disposed therein.

(22) As CBMs **58x** may execute within physical components, CBMs typically are natively conversant in one or more corresponding native I/O types, such as analog input/output, Fieldbus, Railbus, etc. Further, CBMs **58x** typically have no knowledge as to whether they are executing on a Virtual Node **30x** of the virtual plant environment **12**, on a Physical Node **40x** disposed within the physical plant environment **15**, or on another physical component disposed within the physical plant environment **15**, e.g., nodes PNa-PNk. As such, a CBM **58x** sends and receives data in a same manner and using the same processing logic and I/O type irrespective of whether the CBM **58x** is executing in the virtual plant environment **12** or in the physical plant environment **15**.

(23) In the embodiment of the Virtual Node **52a**, process-related payload data that is sent and received by the CBM **58a** is abstracted for delivery to/from the VN **52a** via a Virtual Process I/O (PIO) Subsystem **60**. Generally speaking, the Virtual PIO Subsystem **60** simulates native, physical PIO delivery for the Virtual Node **52a**. That is, during run-time of the Virtual Node **52a**, the Virtual PIO Subsystem **60** communicates I/O process values and events to/from the CBM **58a** using its native I/O, e.g., as though the I/O process values and events were originating from/being delivered to physical hardware. As such, the Virtual PIO Subsystem **60** may be tailored for the specific type of the Virtual Node **52a**, where the Virtual Node type is governed by its CBM **58a**. For example, if the VN **52a** represents a physical controller that communicates with other devices using Railbus I/O, then the Virtual PIO Subsystem **60** provides I/O data to/from the control routine CBM **58a** in the form that is utilized by Railbus I/O cards. If the VN **52a** represents a CIOC, then the Virtual PIO Subsystem **60** provides I/O data to/from the remote I/O CBM **58a** in the form that is utilized by CHARMS.

(24) Further, the Virtual PIO Subsystem **60** may handle data publication and subscription on behalf of the Virtual Node **52a**. That is, the Virtual PIO Subsystem **60** may publish data generated by the CBM **58a** to the Pub/Sub Layer **55a**, and the Virtual PIO Subsystem **60** may subscribe to data generated by other nodes (and forwarded by the I/O Switch **25**) via the Pub/Sub Layer **55a**. In an embodiment, subscriptions to and publications of data may be based on a tag or other identifier that is unique within the MPDSC platform **10**. The tag or other identifier may uniquely identify data, a node, a device, or a component of the MPDSC platform **10**, for example. In an embodiment, the tags and/or identifiers may be assigned during configuration and/or commissioning.

(25) As such, at each virtual node **52a** within the virtual environment **12**, the Virtual PIO Subsystem **60** maintains a logical separation between the CBM **58a** and corresponding physical I/O (e.g., via Pub/Sub Layer **55a**, virtual communication network **45**, and other Pub/Sub Layers **32**, **35**, **38**, **42**), similar to the logical separation maintained within the physical environment **15** at each physical node having an on-board CBM **58a** and its corresponding physical I/O (e.g., via a physical I/O card or marshalled I/O arrangement). Generally speaking, the Virtual PIO subsystem **60** of any Virtual Node **30** maintains a logical separation between any I/O subscriber (the CBM **58a**, another module, another type of I/O consumer, etc.) that is included in the Virtual Node **30** and its corresponding physical I/O.

(26) In the embodiment of the Virtual Node **52b** shown in FIG. 2, the Virtual PIO Subsystem **60** is omitted or is not utilized. Typically, the types of virtual nodes that utilize the VN architecture **52b** are those nodes whose CBMs **58b** are natively able to communicate with the physical components and technology via which the virtual communication network **45** is implemented. For example, if the virtual communication network **45** is implemented via an IP protocol over Ethernet, then a Virtual Node **52b** that simulates an EIOC includes a CBM **58b** that is natively configured to communicate using IP protocol over Ethernet. Consequently, the EIOC Virtual Node **52b** may exclude (or may turn off or ignore the operation of) the Virtual PIO Subsystem **60**, and the EIOC Virtual Node **52b** may handle delivery of data to/from the Virtual Node of **52b** by using only the CBM **58b** and the Pub/Sub Layer **55b**.

(27) Physical Nodes

(28) FIG. 3 is a block diagram of an example physical plant environment **100** in conjunction with

the MPDSC platform **10** of FIG. **1** may operate. For example, the physical environment portion **15** of the MPDSC platform **10** illustrated in FIG. **1** may include at least portions of the physical plant environment **100**.

(29) The physical plant **100** controls an industrial process (or, in some embodiments, operates in conjunction with a virtual plant environment, such as the virtual plant environment **12**, to control the industrial process), where the industrial process may be said to have one or more “process outputs” characterizing the state of the process (e.g., tank levels, flow rates, material temperatures, etc.) and one or more “process inputs” (e.g., the state of various environmental conditions and actuators, the manipulation of which may cause process outputs to change). The physical process plant **100** of FIG. **3** includes a field environment **102** and a back-end environment **105**, each of which is communicatively connected to the other by a process control backbone or data highway **108**, which may include one or more wired and/or wireless communication links and/or networks, and may be implemented using any desired or suitable communication protocol such as, for example, an Ethernet protocol, an IP protocol, or another packet protocol.

(30) At a high level (and as shown in FIG. **3**), the field environment **102** includes physical components (e.g., process control devices, networks, network elements, etc.) that are installed and interconnected to operate to control the industrial process during run-time. By and large, these physical components are located, disposed, or otherwise included in the field environment **102** of the physical process plant **100** in which raw materials are received and processed using the physical components disposed therein to thereby generate one or more physical products (e.g., paper, refined oil, pharmaceuticals, etc.). By contrast, the back-end environment **105** of the physical process plant **100** includes various physical components such as computing devices, operator workstations, databases or databanks, etc. that are shielded and/or protected from the harsh conditions and materials of the field environment **102**. In some configurations, various computing devices, databases, and other components and equipment included in the back-end environment **105** of the physical process plant **100** may be physically located at different physical locations, some of which may be local to the physical plant **100**, and some of which may be remote.

(31) As shown in FIG. **3**, the field environment **102** includes one or more process controllers **110** that are communicatively connected to the data highway **108**. Each of the process controllers **110** may be connected to one or more intermediary nodes **112**, **115** (e.g., I/O cards, I/O devices, I/O systems, etc.) facilitating communication between the controllers **110** and the field devices. Generally speaking, in the process control industry, the term “I/O” is sometimes used in a number of related but different contexts. The term generally refers to a logical link or communication channel that communicatively couples a field device to an I/O card or controller (e.g., “I/O channel”), but may be used when referring to a number of other concepts, such as the physical devices that are utilized to transmit signals to or receive signals from field devices via I/O channels (e.g., “I/O devices” or “I/O cards”) and connectors or terminals associated with the I/O devices (e.g., “I/O connectors”). I/O devices and I/O cards **112**, **115** may be stand-alone, individual physical devices each of which is connected to a respective controller and to one or more respective field devices, such as illustrated in FIG. **3**. In some arrangements (not shown in FIG. **3**), I/O devices, cards, connectors, and other I/O-related components such as terminal blocks, modules, processors, etc. are included in an I/O electronic marshalling system that enables flexible I/O delivery between multiple controllers and multiple field devices of various types, such as described in U.S. Pat. Nos. 7,684,875; 8,332,567; 8,762,618; 8,977,851; 9,083,548; 9,411,769; 9,495,313; and 9,946,240, the entire contents of which are expressly incorporated by reference herein. As such, for clarity of discussion and as utilized herein, the term “I/O devices” refers generally to physical I/O devices, cards, electronic marshalling systems, and components thereof via which I/O channels are implemented to thereby communicatively couple field devices and controllers.

(32) Still, in the process control industry, the term “I/O” generally may be used to refer to the signals transmitted on the I/O channel (e.g., “I/O signals”), variables or commands represented by

the signals (e.g., “I/O parameters”), or to the values of the variables or commands carried by the signals (e.g., “I/O parameter values” or “I/O data payload”). Accordingly, for clarity of discussion and as utilized herein, I/O signals, I/O parameters, and I/O parameter values are collectively and generally referred to herein as “I/O data” or “process I/O data.”

(33) To the extent the term “I/O” is referenced herein without a qualifier, the context of the sentence should make clear which of these concepts is being discussed. Further, it should be understood that an “I/O channel” represents a particular type of “communication channel” or “channel.” That is, unless the context of the sentence suggests otherwise, references in this description to the term “channel” or the term “communication channel,” without the qualifier “I/O,” may refer to a communication link that could be an I/O channel in some implementations, but may also refer to a communication link other than an I/O channel in some implementations.

(34) At any rate, and returning to FIG. 3, each process controller **110** of the physical process plant **100** implements a control strategy defined by one or more control routines (e.g., one or more component behavior modules), which may be stored in a memory of the controller **110**. When a processor of the controller executes one or more of the control routines, the controller transmits to a field device a control signal (i.e., a “control output”) over wired or wireless process control communication links or networks to other field devices to control the operation of a process in the plant **100**. The controller may generate a control signal based on: (i) one or more received signals, which may be referred to as “control inputs” (e.g., one or more received signals representing measurements obtained by field devices), and (ii) the logic of the one or more control routines, which may be defined by one or more software elements (e.g., function blocks). Typically, a controller manipulates a process input (which may be referred to as a “manipulated variable”) to change a particular process output (which may be referred to as a “controlled variable” or simply a “process variable”) based on feedback (i.e., a measurement of the controlled variable) and a desired value for the process output (i.e., a setpoint).

(35) Generally, at least one field device performs a physical function (e.g., opening or closing a valve, increasing or decreasing a temperature, taking a measurement, sensing a condition, etc.) to control the operation of a process implemented in the physical process plant **100**. Some types of field devices communicate with controllers by using I/O devices. Process controllers, field devices, and I/O devices may be wired or wireless, and any number and combination of wired and wireless process controllers, field devices, and/or I/O devices may be included in the process plant environment or platform **100**.

(36) The Front-End Environment **102** of the Plant **100**

(37) For example, FIG. 3 illustrates a process controller **110** that is communicatively connected to wired field devices **125-132** via input/output (I/O) devices **112** and **115**, and that is communicatively connected to wireless field devices **140-146** via a wireless gateway **168** and the data highway **108**. In some configurations (not shown), the controller **110** may be communicatively connected to the wireless gateway **168** using one or more communications networks other than the backbone **108**, such as by using any number of other wired or wireless communication links that support one or more communication protocols, e.g., Wi-Fi or other IEEE 802.11 compliant wireless local area network protocol, mobile communication protocol (e.g., WiMAX, LTE, or other ITU-R compatible protocol), Bluetooth®, HART®, WirelessHART®, Profibus, FOUNDATION® Fieldbus, etc.

(38) The controller **110**, which may be, by way of example, the DeltaV™ controller sold by Emerson Process Management, may operate to implement a batch industrial process or a continuous industrial process using at least some of the field devices **125-132** and **140-146**. In an embodiment, in addition to being communicatively connected to the process control data highway **108**, the controller **110** is also communicatively connected to at least some of the field devices **125-132** and **140-146** using any desired hardware and software associated with, for example, standard 4-20 mA devices, I/O devices **112**, **115**, and/or any smart communication protocol such as the

FOUNDATION® Fieldbus protocol, the HART® protocol, the WirelessHART® protocol, etc. In FIG. 3, the controller **110**, the field devices **125-132** and the I/O devices **112, 115** are wired devices, and the field devices **140-146** are wireless field devices. Of course, the wired field devices **125-132** and wireless field devices **140-146** could conform to any other desired standard(s) or protocols, such as any wired or wireless protocols, including any standards or protocols developed in the future.

(39) The process controller **110** of FIG. 3 includes a processor **120** that implements or oversees one or more process control routines **118** (e.g., that are stored in a memory **122** of the controller **110**). The processor **120** is configured to communicate with the field devices **125-132** and **140-146** and with other nodes communicatively connected to the controller **110**. It should be noted that any control routines or modules described herein may have parts thereof implemented or executed by different controllers or other devices if so desired. Likewise, the control routines or modules **118** described herein which are to be implemented within the process control plant **100** may take any form, including software, firmware, hardware, etc. Control routines may be implemented in any desired software format, such as using object oriented programming, ladder logic, sequential function charts, function block diagrams, or using any other software programming language or design paradigm. The control routines **118** may be stored in any desired type of memory **122**, such as random access memory (RAM), or read only memory (ROM). Likewise, the control routines **118** may be hard-coded into, for example, one or more EPROMs, EEPROMs, application specific integrated circuits (ASICs), or any other hardware or firmware elements. Thus, the controller **110** may be configured to implement a control strategy or control routine in any desired manner.

(40) The controller **110** implements a control strategy using what are commonly referred to as function blocks, where each function block is an object or other part (e.g., a subroutine) of an overall control routine and operates in conjunction with other function blocks (via communications called links) to implement process control loops within the MPDSC platform **10**. Control based function blocks typically perform one of: (i) an input function, such as that associated with a transmitter, a sensor or other process parameter measurement device (sometimes referred to as “input blocks”); (ii) a control function, such as that associated with a control routine that performs PID, fuzzy logic, etc. (sometimes referred to as “control blocks”); or (iii) an output function which controls the operation of some device, such as a valve, to perform some physical function within the process control plant **100** (sometimes referred to as “output blocks”). Of course, hybrid and other types of function blocks exist.

(41) Function blocks may be stored in and executed by the controller **110**, which is typically the case when these function blocks are used for, or are associated with standard 4-20 mA devices and some types of smart field devices such as HART® devices, or may be stored in and implemented by the field devices themselves, which can be the case with FOUNDATION® Fieldbus devices. One or more of the control routines **118** may implement one or more control loops which are performed by executing one or more of the function blocks. In a sense, the control routines **118** may be viewed as the component behavior modules of the controller **110**.

(42) The wired field devices **125-132** may be any types of devices, such as sensors, valves, transmitters, positioners, etc., while the I/O devices **112** and **115** may be any types of process control I/O devices conforming to any desired communication or controller protocol. For example, the I/O devices **112, 115** may be included in an I/O electronic marshalling system. In FIG. 3, the field devices **125-128** are standard 4-20 mA devices or HART® devices that communicate over analog lines or combined analog and digital lines to the I/O device **112**, while the field devices **129-132** are smart devices, such as FOUNDATION® Fieldbus field devices, that communicate over a digital bus to the I/O device **115** using a FOUNDATION® Fieldbus communications protocol. In some embodiments, though, at least some of the wired field devices **125, 126** and **128-131** and/or at least some of the I/O devices **112, 115** additionally or alternatively communicate with the controller **110** using the process control data highway **108** and/or by using other suitable control system

protocols (e.g., Profibus, DeviceNet, Foundation Fieldbus, ControlNet, Modbus, HART, etc.). In some arrangements (not shown in FIG. 3), at least some of the field devices **125-132** may communicate with the controller **110** via an electronic I/O marshalling system instead of via an individual I/O device **112, 115**.

(43) In FIG. 3, the wireless field devices **140-146** communicate via a wireless process control communication network **165** using a wireless protocol, such as the WirelessHART® protocol. Such wireless field devices **140-146** may directly communicate with one or more other devices or nodes of the wireless network **165** that are also configured to communicate wirelessly (using the wireless protocol or another wireless protocol, for example). To communicate with one or more other nodes that are not configured to communicate wirelessly, the wireless field devices **140-146** may utilize a wireless gateway **168** connected to the process control data highway **108** or to another process control communications network. The wireless gateway **168** provides access to various wireless devices **140-158** of the wireless communications network **165**. In particular, the wireless gateway **168** provides communicative coupling between the wireless devices **140-158**, the wired devices **125-132**, and/or other nodes or devices of the physical process control plant **100**. For example, the wireless gateway **168** may provide communicative coupling by using the process control data highway **108** and/or by using one or more other communications networks of the physical process plant **100**.

(44) Similar to the wired field devices **125-132**, the wireless field devices **140-146** of the wireless network **165** perform physical control functions within the physical process plant **100**, e.g., opening or closing valves, or taking measurements of process parameters. The wireless field devices **140-146**, however, are configured to communicate using the wireless protocol of the network **165**. As such, the wireless field devices **140-146**, the wireless gateway **168**, and other wireless nodes **152-158** of the wireless network **165** are producers and consumers of wireless communication packets.

(45) In some configurations of the physical process plant **100**, the wireless network **165** includes non-wireless devices. For example, in FIG. 3, a field device **148** of FIG. 3 is a legacy 4-20 mA device and a field device **150** is a wired HART® device. To communicate within the network **165**, the field devices **148** and **150** are connected to the wireless communications network **165** via a wireless adaptor **152a, 152b**. The wireless adaptors **152a, 152b** support a wireless protocol, such as WirelessHART, and may also support one or more other communication protocols such as Foundation® Fieldbus, PROFIBUS, DeviceNet, etc. Additionally, in some configurations, the wireless network **165** includes one or more network access points **155a, 155b**, which may be separate physical devices in wired communication with the wireless gateway **168** or may be provided with the wireless gateway **168** as an integral device. The wireless network **165** may also include one or more routers **58** to forward packets from one wireless device to another wireless device within the wireless communications network **165**. In FIG. 3, the wireless devices **140-146** and **152-158** communicate with each other and with the wireless gateway **168** over wireless links **160** of the wireless communications network **165**, and/or via the process control data highway **108**.

(46) The Back-End Environment **105** of the Plant **100**

(47) As noted, the back-end environment **105** may include various components such as computing devices, operator workstations, databases or databanks, etc. that are typically shielded and/or protected from the harsh conditions and materials of the field environment **102**. For example, the back-end environment **105** may include any one or more of the following, each of which may be communicatively connected to the data highway **108**: (i) one or more operator workstations **170a** and other local or remote user interface devices **170b**; (ii) one or more configuration applications **172a** and configuration databases **172b**; (iii) one or more other types of applications **175a** and/or databases **175b**, which may include, for example, tools, diagnostics, asset management systems, simulators, and/or other types of applications; (iv) one or more other wireless access points **178** that communicate with other devices associated with plant **100** (e.g., user interface devices **170b** or other devices) using various wireless protocols; (v) one or more plant gateway systems **180** to other

plants; (vi) one or more edge gateway systems **182** to systems **185** that are external to the immediate, OT layers of the physical process control platform **100** (e.g., IT networks/systems of the enterprise, and/or external data networks/systems, which may be implemented on cloud computing and/or other suitable platforms); and/or (vii) other physical components that are specially configured via hardware and software to support the process plant **100**.

(48) Operator Workstations and User Interface Devices **170a**, **170b**

(49) The operator workstations **170a** and other user interface devices **172b** may be utilized by operators to view and monitor run-time operations of the physical process plant **100**, as well as to take any diagnostic, corrective, maintenance, and/or other actions that may be required. At least some of the operator workstations **172a** may be located at various, protected areas in or near the plant **100**, and in some situations, at least some of the operator workstations **172b** may be remotely located, but nonetheless in communicative connection with the plant **100**. Operator workstations **172a**, **172b** may be wired or wireless computing devices.

(50) Configuration Applications **172a** and Configuration Databases **172b**

(51) The configuration applications **172a** and the configuration databases **172b** may be utilized to configure certain aspects of the plant **100**, such as control modules/routines, user interfaces, data monitoring/analytics, etc. For example, various instances of the configuration application **172a** may execute on one or more computing devices (not shown) to enable users to create or change process control modules and download these modules via the data highway **108** to the controllers **110**, to create or change operator interfaces via which in operator is able to view data and change data settings within process control routines, to create or change data monitoring/analytics routines and functions which may be downloaded into various physical components within the field environment **102**, etc. The configuration databases **172b** store the created (e.g., configured) modules, operator interfaces, data monitoring/analytics, etc. Generally, the configuration applications **172a** and configuration databases **172b** are centralized and have a unitary logical appearance to the physical process control platform **100**, although multiple instances of the configuration applications **172a** may execute simultaneously within the physical process control platform **100**, and the configuration databases **172b** may be implemented across multiple physical data storage devices. Accordingly, the configuration applications **172a**, the configuration databases **172b**, and the user interfaces thereto (not shown) comprise a configuration or development system **172** for various types of modules, e.g., control modules, display or operator interface modules, and/or analytics modules. Typically, but not necessarily, the user interfaces for the configuration system **172** are different than the operator workstations/user interface devices **170**, as the user interfaces for the configuration system **172** are utilized by configuration and development engineers irrespective of whether or not the plant **100** is operating in production mode, whereas the operator workstations/user interface devices **170** are utilized by operators during run-time operations of the physical process plant **100**.

(52) Regarding commissioning of the physical components of the MPDSC platform **100**, the configuration database **172b** may store data and other information that specifically identifies and/or addresses the various physical devices or components and their interconnections that are planned for or desired to be implemented on the process plant floor or field environment **102**. Some of this commissioning data may be provided to components in the field environment **102** for use in commissioning of devices and loops therein, and some of this data may be utilized in the back-end environment **105**, e.g., for the design, development, and preparation of control modules, operator interface modules, and/or data analytics modules that will operate in conjunction with the field environment **102** during live operations of the physical process plant **100**. In an example, an approved control module is downloaded from the configuration database **172b** into a process controller **110** so that, when executed during live operations, the process controller **110** operates in accordance with its resident control module to send and receive various signals to/from other components in its loop (and, in some cases, to/from other process controllers), thereby controlling

at least a portion of the process in the physical process plant **100**.

(53) The configuration database **172b** may store a number of logical identifiers of components in the field environment **102**, enabling the controller **110** and other devices to reference the components and signals associated with the components by way of the logical identifiers. For example, for a given field device, the configuration database **172b** may store information mapping or binding a logical identifier to a particular hardware address or I/O channel. The hardware address may identify a particular controller, a particular I/O device connected to the particular controller, and/or a particular address for the I/O channel connecting the particular I/O device to the field device. In some instances, this mapping or binding may be stored at the controller **110**, the user interface device **170b**, the operator workstation **170a**, or any other desired device (e.g., any device needing to resolve the logical identifier). After a logical identifier has been bound to a hardware address or I/O channel, the identifier is considered “assigned.” In some cases, the plant **100** includes “unassigned” logical identifiers, which are identifiers that a software element (e.g., a control routine and/or a function block) references but that has no binding. That is, a logical identifier is considered “unassigned” when the plant **100** and the configuration database **172b** have no hardware address or I/O channel that has been bound to the tag. Thus, when an unassigned logical identifier is referenced by a control routine, no value carried by a signal in the plant **100** will be read and no command will be transmitted via a signal to a field device in the plant **100**.

(54) Examples of such logical identifiers include Device Tags (DTs), each of which represents a particular instrument, controller, valve, or other physical field device, and Device Signal Tags (DSTs), each of which represents a particular signal that is received or generated by a particular device and that typically corresponds to a particular parameter utilized by the field device. For some devices, a Device Signal Tag comprises a combination of a device's Device Tag and an identifier of a specific signal received or generated by that device, e.g., an identifier of a specific parameter referenced by a control module. For some devices, typically legacy or dumb devices, a Device Tag represents both the physical device and a signal generated by the device. Generally speaking, a device's logical identifier is used by the physical process plant **100** in both the field environment **102** and in the back-end environment **105** to uniquely identify the device. The DTs and DSTs may be referred to as “system tags” or “system identifiers.”

(55) In some instances, the smart field devices **129-132** also may store logical identifiers unique to the smart field devices **129-132**. These logical identifiers may be distinct from the system tags utilized by the plant **100** to identify the field devices **129-132**, and may be referred to as “source identifiers” or “source tags.” Source tags may or may not be stored at the configuration database **172b**, depending on the implementation.

(56) The configuration database **172b** may store data and other information that specifically identifies and/or address various virtual devices or components (e.g., various Virtual Nodes **30**) that are planned for or desired to be implemented in the virtual plant environment **12**. Similar to physical devices and components, the configuration database **172** may store respective logical identifiers of components of the virtual environment **12**, thereby enabling other devices and/or components (whether physical or virtual) to reference the virtual components. For example, similar to Physical Nodes **40x**, various Virtual Nodes **30x** may be uniquely identified within the MPDSC platform **10** (e.g., across both the virtual **12** and physical **15** environments) via their respective Device Tags (DTs), Devices Signal Tags (DSTs), source tags, and/or other types of unique identifiers. Further sections of this disclosure discuss the configuring and identification of virtual devices and components in more detail.

(57) Other Applications **175a** and Databases **175b**

(58) The other applications **175a** and databases **175b** may include instances of applications that are specific to the process plant **100**, such as diagnostic applications/databases, data historian applications/databases, system level health monitoring applications/databases, local and/or remote user interfaces, etc. Generally speaking, the other applications **175a** and databases **175b** may

include one or more applications executing at the OT layers **18** of the process plant **100** and their associated databases. Additionally or alternatively, the other applications **175a** and databases **175b** may include one or more applications executing at the IT layers **20** associated with the process plant **100**, such as inventory management applications, personnel management applications, supply chain management applications, other types of enterprise-related applications, weather/environmental applications, etc., and their associated databases. For example, the other applications **175a** and databases **175b** may include the applications **21a-21n** and their associated databases. Still further additionally or alternatively, the other applications **175a** and databases **175b** may include one or more applications that are external to and/or that execute remotely from the process plant **100** and/or its enterprise, such as simulation applications, analytics applications, IoT applications, IIoT applications, etc., and their associated databases. For example, the other applications **175a** and databases **175b** may include the applications **22a-22m** and their associated databases. The other applications **175a** and databases **175b** may include third-party applications and/or may include applications provided by the enterprise associated with the process plant **100**. At least some of the other applications **175a** and databases **175b** may be accessed via an edge gateway system **28** and/or some other type of security and/or firewall system.

(59) Wireless Access Points 178

(60) The one or more other wireless access points **178** enable devices in the back-end environment **105** (and sometimes in the field environment **102**) to communicate with other devices using wireless protocols, such as Wi-Fi or other IEEE 802.11 compliant wireless local area network protocols, mobile communication protocols such as WiMAX (Worldwide Interoperability for Microwave Access), LTE (Long Term Evolution) or other ITU-R (International Telecommunication Union Radio communication Sector) compatible protocols, short-wavelength radio communications such as near field communications (NFC) and Bluetooth, or other wireless communication protocols. Typically, such wireless access points **178** allow handheld or other portable computing devices (e.g., user interface devices **170b**) to communicate over a respective wireless process control communication network that is different from the wireless network **165** and that supports a different wireless protocol than the wireless network **165**. For example, a wireless or portable user interface device **170b** may be a mobile workstation or diagnostic test equipment that is utilized by an operator within the physical process plant **100** (e.g., an instance of one of the operator workstations **170a**). In some scenarios, in addition to portable computing devices, one or more process control devices (e.g., controller **110**, field devices **125-132**, or wireless devices **168, 140-158**) also communicate using the wireless protocol supported by the access points **174**.

(61) Gateway Systems 180, 182

(62) The gateway nodes or systems **180** and **182** may interface with systems that are external to the immediate physical process control plant **100**. Typically, such systems are customers or suppliers of information generated or operated on by the physical process plant **100**. For example, the process control plant **100** may include a plant gateway node **180** to communicatively connect the immediate physical process plant **100** with another process plant. Additionally or alternatively, the process control plant **100** may include an edge gateway node or system **182** to communicatively connect the immediate physical process plant **100** with an external public or private system, such as a laboratory system (e.g., Laboratory Information Management System or LIMS), an operator rounds database, a data consolidation and viewing system, a data analytics system, a materials handling system, an asset management system, a maintenance management system, a product inventory control system, a production scheduling system, a weather data system, a shipping and handling system, a packaging system, the Internet, an IOT application, an IIOT application, or other external systems.

(63) Generally speaking, the edge gateway system **182** allows communications to be securely delivered between the process plant **100** and other networks **185**. In an example architecture, an

edge gateway system **182** includes an internal- or plant-facing engine (which may be referred to as a “field gateway”) and an external- or outwards-facing engine (which may be referred to as an “edge gateway”), where the plant-facing engine and the external-facing engine cooperate to securely deliver process plant data (e.g., data generated by the OT layers) to external networks and/or applications (e.g., IT layers and/or external networks and/or systems) that are consumers of the plant data. In one of many embodiments, the plant-facing engine collects data generated by various components of the process plant **100**, and securely transmits the collected data across one or more secured communication links to the external-facing engine, e.g., via a first set of publications. The external-facing engine has subscribed to the various publications generated by the plant-facing engine and obtains the collected plant data therefrom. In turn, the external-facing engine securely transmits the obtained plant data to one or more external client applications (e.g., applications **21a-21n**, **22a-22m**) within the networks **185**, e.g., via a second set of publications. Publications and subscriptions at the plant-facing engine and/or at the external-facing engine may be respectively configured as desired. Typically, though, the publication/subscription, encryption, and/or other secure data delivery mechanisms that are utilized between the plant-facing engine and the external-facing engine differ from the publication/subscription, encryption, and/or other secure data delivery mechanism that are utilized between the external-facing engine and external applications. In some embodiments, for additional security, the edge gateway system **182** includes a one-way data diode disposed between the plant-facing engine and the external-facing engine to prevent data from flowing from external networks **185** into the process plant **100**. Examples of edge gateway systems may be found in U.S. Patent Publication Numbers 20180115516; 20180115528; 20180115517; and 20180113442, the entire disclosures of which are hereby incorporated by reference. Of course, other edge gateway systems may be additionally or alternatively utilized by the process plant **100**.

(64) It is noted that although FIG. **3** only illustrates a single controller **110** with a finite number of field devices **125-132** and **140-146**, wireless gateways **168**, wireless adaptors **152**, access points **155**, routers **158**, and wireless process control communications networks **165** included in the example physical process plant **100**, this is only an illustrative and non-limiting embodiment. Any number of controllers **110** may be included in the process control plant or platform **100**, and any of the controllers **110** may communicate with any number of wired or wireless devices and networks **125-132**, **140-146**, **168**, **152**, **155**, **158** and **165** to control a process in the plant **100**.

(65) Now jointly referring to FIGS. **1** and **3** with respect to Physical Nodes, each Physical Node **40a-40c** and PNa-PNk shown in FIG. **1** may be a respective one of the physical components or devices discussed with respect to the process control plant **100** of FIG. **3**. For example, PN **40b** may be a process controller that has a respective Pub/Sub Layer **42b** and that is communicatively connected to a field device PNh via an I/O card PNg. In another example, PN **40c** may be a CIOC (having a respective Pub/Sub Layer **42c**) to which six different field devices PNa-PNf are communicatively connected, or PN **40c** may be a wireless access point **178** or an EIOC (having a respective Pub/Sub Layer **40c**) to which various other physical components PNa-PNf without Pub/Sub Layers are communicatively connected. In yet another example, PN **40a** may be an I/O marshalling device or I/O hub device that delivers, via its Pub/Sub layer **42a**, process I/O data on behalf of multiple other PNs that do not have Pub/Sub layers, e.g., PNi-PNk. The I/O marshalling or hub device **40a** may be a stand-alone device, or may be included in another physical component disposed within the physical plant environment **15**, such as in a controller **110**, an I/O device **112**, **115** (e.g., a CIOC, an ECOC, a WIOC, etc.), an access point **178**, a gateway **180**, **182**, etc. Further, in some implementations, the edge gateway system **28** and its Pub/Sub Layer **38** may be a Physical Node that is communicatively connected (not illustrated in FIG. **1**) to various physical components of the physical plant environment **15** (e.g., PNs **40a-40c** and PNa-PNk), e.g., in a manner such as discussed with respect to FIG. **3**.

(66) I/O Switch

(67) As mentioned above, the I/O Switch **25** shown in FIG. **1** serves as a data broker, switching fabric, or router for process I/O data between nodes of the virtual communication network **45**. That is, the I/O Switch **25** forwards, on behalf of a publishing node, I/O data generated by the publishing node to nodes that have subscribed to that data. Generally speaking, the I/O Switch **25** may maintain (and update) records of what I/O data is published by which node, and which nodes have subscribed to which published I/O data. In an embodiment, the I/O data may be identified within the records by the unique identifier, name, or tag that is unique across the MPDSC platform **10**, or by a respective indication thereto. The unique identifier may identify the process-related payload data included in the I/O data and/or its publishing node, for example. In an embodiment, the unique identifiers, names, or tags are assigned during configuration and/or commissioning. The I/O Switch **25** utilizes the stored records to route incoming, published data to subscribers of the published data. For example, the I/O Switch **25** may subscribe to I/O data that is published by various nodes **28**, **30x**, **40x**, and upon receiving the published I/O data via its subscriptions, the I/O Switch **25** may itself publish the received I/O data to selected subscriber nodes **28**, **30x**, **40x** in accordance with the stored records. Typically, but not necessarily, the I/O Switch **25** does not maintain, record, or store any I/O data and/or process-related payload data that is published by other nodes of the virtual communication network **45**.

(68) Importantly, the I/O Switch **25** is configured to switch or forward received process I/O data with minimal delay. Indeed, in a prototype, the I/O Switch **25** was able to forward received process I/O data through the I/O Switch **25** in time intervals measured in hundreds of microseconds, such as under 500 microseconds, under 200 microseconds, and under 100 microseconds, depending on loading.

(69) Generally speaking, the I/O Switch **25** may be implemented via hardware, firmware, and/or software. In an example, the I/O Switch **25** is implemented using one or more physical, hardware devices on which specialized software is installed to provide at least some of the I/O Switch **25** functionality described herein; that is, the I/O Switch **25** may be implemented as an appliance. In another example, the I/O Switch **25** is implemented as software (e.g., programs, routines, applications, etc.) that may be installed on and executed by a server or bank of computing devices to provide at least some of the I/O Switch **25** functionality described herein. In yet another example, the I/O Switch **25** is implemented via virtualization, e.g., as a virtual machine, a container (such as Docker, LXD, etc.), or another type of virtualized implementation.

(70) The Pub/Sub Layer **35** of the I/O Switch **25** generally is similar in configuration and functionality to the Pub/Sub Layers **32x**, **38**, **42x**, **55x** described elsewhere within this disclosure. However, the Pub/Sub Layer **35** of the I/O Switch **25** is further configured to accept and maintain subscription requests for various identified process I/O data. As discussed above, subscriptions are recorded at and maintained by the I/O Switch **25**, e.g., in one or more tangible, non-transitory memories. Further, the Pub/Sub Layer **35** of the I/O Switch **25** is configured to forward published process I/O data to subscribing nodes, which may include other I/O Switches.

(71) To illustrate, FIG. **4** depicts an example arrangement **200** of multiple I/O Switches **202**, **205**, **208**, **210** which may be included in the MPDSC platform **10** of FIG. **1**. In an example, each I/O Switch **202**, **205**, **208**, **210** has a respective architecture similar to that of the I/O Switch **25**, including a respective Pub/Sub Layer (as indicated by the respective hash-marked portions). Also similar to the I/O Switch **25**, each I/O Switch **202**, **205**, **208**, **210** is associated and interconnected with, via the virtual communication network **225**, a respective set of VNs and/or PNs **212a-212n**, **215a-215m**, **218a-218p**, and **220a-220q** amongst which the respective I/O Switch **202**, **205**, **208**, **210** forwards process I/O data via publishing and subscribing, such as in a manner described above. Although the arrangement **200** includes four I/O switches **202**, **205**, **208**, **210**, the concepts discussed herein with respect to the arrangement **200** are easily applied to greater or lesser numbers of I/O switches that are interconnected in a mesh topology.

(72) In FIG. **4**, each I/O Switch **202**, **205**, **208**, **210** is further able to forward, via the virtual

communication network **225**, process I/O data to each of the other I/O Switches **202, 205, 208, 210**. By interconnecting multiple I/O Switches **202, 205, 208, 210**, the number of serviced Virtual Nodes and/or Physical Nodes is able to be expanded to support larger systems **10** and to provide additional delivery bandwidth. In the arrangement **200**, each I/O Switch **202, 205, 208, 210** may publish data on behalf of its respective set of VNs/PNs to other I/O Switches **202, 205, 208, 210**. Similarly, each I/O Switch **202, 205, 208, 210** may subscribe to data that is forwarded by other I/O Switches **202, 205, 208, 210** on behalf of their respective sets of VNs/PNs **212a-212n, 215a-215m, 218a-218p, 220a-220q**.

(73) The arrangement **200** illustrates each of the I/O Switches **202, 205, 208, 210** as being directly linked to each of the other I/O Switches **202, 205, 208, 210**. That is, the virtual communication network **225** has a fully-interconnected or mesh topology. Accordingly, in an implementation of this arrangement **200**, the maximum transmission delay from a publisher node to a subscriber node is bounded, as the maximum number of hops that a particular process I/O data payload may undergo is three. In other implementations of the arrangement **200**, the maximum number of hops may differ. For example, the maximum number of hops may be increased, e.g., for monitoring traffic, when time-sequencing capabilities, such as Time Sequencing Networking, or other suitable time-sequencing capabilities) are utilized, and the like. A maximum number of hops within the arrangement **200** may be configured and modified, if desired.

(74) Of course, other interconnected topologies (e.g., hub-and-spoke, star, ring, tree, hybrid, etc.) may be possible in addition to or as an alternative to the mesh topology for the virtual communication network **225**. Generally speaking, subscriptions to process I/O data that is published by a particular publishing node, e.g., node **215a**, are made to and managed by its corresponding I/O Switch, e.g., I/O Switch **205**. However, if the I/O Switch **205** does not have a record for the particular data to which a subscription is requested, the I/O Switch **205** may forward the subscription request to the other I/O Switches **202, 208, 210**. The particular I/O Switch corresponding to the node that publishes the requested data, e.g., I/O Switch **210** corresponding to publishing node **220q**, will have a record corresponding to the requested data and its publishing node **220q**, and will respond to the I/O Switch **205** corresponding to the requesting node **215a**, upon which the I/O Switch **105** may create and store a corresponding record for the requested data. In this manner, a traversal route for the requested data from publishing node **220q** to subscribing node **215a** via the I/O Switches **210, 205** may be established.

(75) In an embodiment, the arrangement **200** may be implemented across multiple MPDSC systems **10**. For example, I/O Switch **202** may be included in a first MPDSC system, and I/O Switch **205** may be included in a different MPDSC system. As such, different MPDSC systems may publish and/or subscribe to each other's process I/O data via one or more links of the virtual communication network **225**.

(76) Real-Time Virtual Control and Associated Operations

(77) As mentioned above, Virtual Nodes **30x** may virtualize the behavior of various physical components which are operable within the physical plant environment **15**. Further, due to the features of the MPDSC platform **10**, virtualized components **30x** of the virtual plant environment **12** may operate in conjunction with physical components of the physical plant environment **15** to perform real-time control of the industrial process plant during run-time to thereby generate physical products from raw materials. Referring simultaneously to FIGS. **1, 2, and 3** to illustrate, the process controller **110** may be virtualized as the Virtual Node **30a**, which has an architecture **52a**. As such, instead of the physical process controller **110** executing its corresponding control modules or routines **118** during run-time, the Virtual Node **110/30a** includes the control modules or routines **118** as its CBM **58a** and executes the corresponding control modules or routines **118** during run-time, which may include sending and receiving signals to/from various field devices disposed within the physical environment **15** via the I/O Switch **25**.

(78) For example, during run-time of the industrial process plant **100**, the virtual controller **110/30a**

may receive data generated by field device **129** via physical I/O device **115**. In this example, physical I/O device **115** is represented in FIG. **1** by physical node PN_g, field device **129** is represented in FIG. **1** by physical node PN_h, and physical node **40b** is an I/O hub device. Data payload generated by field device PN_h is delivered via I/O device PN_g to the I/O hub device **40b**, which is a node of the virtual communication network **45**. The I/O hub device **40b** may publish, to the I/O Switch **25**, the data payload that was generated by the field device PN_h and published by I/O hub device **40b**, to which the I/O Switch **25** has a subscription. In turn, the I/O Switch **25** may publish the payload data generated by field device PN_h to the virtual controller **110/30a**, whereupon the CBM **58a** (e.g., the control modules/routines **118**) of the virtual controller **110/30a** may operate on the payload data generated by the field device PN_h. The CBM **58a** may generate output payload data that is to be delivered to another virtual or physical node via the virtual communication network **45**.

(79) In another example, during run-time of the industrial process plant **100**, the virtual controller **110/30a** may receive data generated by wireless field device **142a**. In this example, physical wireless field device **142a** is represented in FIG. **1** by physical node PN_a, and the physical wireless gateway **168** is represented by PN **40c**. Wireless gateway PN **40c** receives payload data generated by wireless field device PN_a. As PN **40c** is a node of the virtual communication network **45**, PN **40c** publishes, to the I/O Switch **25**, the payload data generated by the wireless field device PN_a to which the I/O Switch **25** has a subscription. In turn, the I/O Switch **25** may publish the payload data generated by the field device PN_a to a virtual node **30b**, where the virtual node **30b** is configured as a representation of an I/O device, such as the I/O device **115**. The virtual I/O device **30b** has subscribed to the payload data generated by the field device PN_a and forwarded by the I/O Switch **25**. As such, the virtual I/O device **30b** obtains the payload data generated by the wireless field device PN_a via its subscription, and publishes anew the obtained payload data generated by the field device PN_a to the I/O Switch **25** for forwarding to appropriate subscribers. The I/O Switch **25** may in turn publish the obtained payload data of the field device PN_a that was published by the virtual I/O device **30b** to its respective subscribers, which (in this example) include the virtual controller **30a**. Upon receipt of the payload data at the virtual controller **30a** via its subscription thereto, the CBM **58a** (e.g., the control modules/routines **118**) of the virtual controller **30a** may operate on the payload data generated by the wireless field device PN_a. The CBM **58a** may generate output payload data that is to be delivered to another virtual or physical node via the virtual communication network **45**.

(80) Virtualization of physical components of industrial process plants provide numerous benefits over industrial process plants that are entirely implemented using physical components. Virtualized components allow an industrial process plant to be flexibly scaled up and/or down in size and number of components with minimal changes to hardware footprint and decreased installation costs. As a virtual nodes may represent a physical device in its entirety or only a portion thereof, control may be easily scaled, e.g., across as many virtual nodes as needed. Additionally, the MPDSC platform **10** provides I/O flexibility via the abstraction of I/O, for example, so that controllers have less dependency on actual configured and downloaded I/O module assignments. That is, at least due to the I/O Switch **25**, I/O binding decisions between physical I/O devices and controllers and field devices may be eliminated. Further, by using virtual physical components, software upgrades to the virtualized physical components is easily achieved. Still further, by using the MPDSC platform **10**, simulation and on-line testing of behavior of physical components is also easily achieved and improved over currently known techniques.

(81) Virtualization Management Node

(82) FIG. **5** is a block diagram illustrating an example architecture supporting the configuration, commissioning, management, and administration of the MPDSC system **10** of FIG. **1**. For purposes of clarity, and not for limitation purposes, FIG. **5** is discussed herein with simultaneous reference to FIGS. **1-4**. As shown in FIG. **5**, a Virtualization Management Node **300** (which is interchangeably

referred to herein as the “VMN 300” or the “Virtual Node 300”) creates, configures, and manages virtualized components of the MPDSC system **10** such as the I/O Switch **25**, various virtual nodes **30x**, the virtual PIO subsystem **60**, the publish/subscribe layers **32x**, **35**, **38**, **42x**, etc. The VMN **300** automatically performs at least some of the creation, configuration, and management of virtual components. Additionally or alternatively, the VMN **300** performs at least some of the creation, configuration, and management of the virtual components based on manual instructions, e.g., manual instructions received via a user interface **302** of the VMN **300**, which may be a local or a remote user interface. Further, the Virtualization Management Node **300** drives and/or coordinates simulation activities of the virtual process environment **12**, which may be performed with and/or without in-line user input during simulations, as is described in more detail below.

(83) Virtual Component Configuration, Administration, and Management

(84) In an embodiment, during the configuration and/or commissioning of the virtual environment **12** and components thereof, the VMN **300** accesses the system configuration database of the plant (e.g., the VMN **300** accesses the configuration database(s) **172b** of the plant **100** via the data highway **108**), and based on the obtained configuration, the VMN **300** determines the types and numbers of virtual nodes, virtual templates and/or subsystems, I/O switches, and/or other virtual components that would best support (or, that are desired to support) the plant configuration. The VMN **300** creates the various types and numbers of virtual nodes **30x**, and configures the I/O Switch **25** and the virtual nodes **30x** to communicate via the virtual communication network **45**, e.g., via respective publish/subscribe layers **32x** and, for some VNs **30x**, via the virtual PIO subsystem **60**. For some plant configurations, the VMN **300** may create and distribute instances of publish/subscribe layers **42x** (and optionally, instances of the virtual PIO subsystem **60**) to various physical components **40x** of the physical plant environment **15**. As such, the VMN **300** may create, configure, and administer any number of nodes **25**, **28**, **30x**, **40x** of the virtual communication network **45**, which may include virtual components that operate in conjunction with physical components of the physical plant to perform real-time control and/or dynamic simulation.

(85) The configuration of the virtual communication network **45** and its nodes **25**, **28**, **30x**, **40x** may be stored in the system configuration database **172b**, and/or may be locally stored in a virtual environment configuration database **305**. In some implementations, a master version of the configuration of the virtual plant environment **12** is stored in the system configuration database **172b** (along with the configuration of the physical plant environment **15**), and a copy of the master configuration of the virtual plant environment **12** is locally stored in the virtual environment configuration database **305**. In some implementations, a master version of the configuration of the virtual plant environment **12** is stored in the virtual environment configuration database **305**, and a copy of the master configuration of the virtual plant environment **12** is stored in the system configuration database **172b** (along with the configuration of the physical plant environment **15**). The VMN **300** coordinates change management and synchronization of data stored in the virtual environment configuration database **305** and related data stored in the system configuration database **172b**.

(86) While the plant **100** is performing operational process control during run-time of the process plant **100**, the VMN **300** may monitor the I/O Switch **25**, virtual nodes **30x**, the virtual communication network **45** and any associated nodes (such as physical nodes **40x**, **28**) with respect to, for example, resource loading, resource availability, resource bandwidth, fault occurrence, and other performance issues of hardware and/or software resources provided by the physical computing platform supporting the virtual environment **12**, e.g., platform of hardware computing devices supporting the virtual environment **12**. Based on detected and/or predicted conditions, the VMN **300** may automatically perform mitigating actions during run-time operations, such as adjusting resource allocations, activating stand-by Virtual Nodes, creating and/or deleting Virtual Nodes, etc., e.g., for load-balancing, fault recovery, and other performance purposes. As an example, the VMN **300** may analyze performance bottlenecks and perform auto-leveling operations

to mitigate any detected bottlenecks. For instance, the VMN **300** may perform auto-leveling at a process data level in response to the amount of virtual network traffic on the virtual communication network **45**, and/or the VMN **300** may perform auto-leveling at a physical computing device level so that CPU and/or physical network utilization is balanced across the physical computing devices, physical data links, and/or physical networks on which the virtual environment **12** is implemented. (87) Administratively, the VMN **300** may perform the saving, snapshot, backup, migration, restoration, etc. of various virtual nodes, virtual templates and subsystems, and associated process data, and the VMN **300** may perform the saving, snapshot, backup, migration, restoration, etc. of the entire virtual environment **12** itself. In an embodiment, each administrative action performed by the VMN **300** may apply to an entirety of the logic simulated by subject Virtual Nodes (e.g., the entirety of the logic performed by collective set of CBMs **58** of the subject Virtual Nodes). For example, when taking and saving a snapshot of a virtual controller, the VMN **300** may not only save data that is received, operated on, generated by, and indicative of the virtual controller and its interconnections and operating states, but the VMN **300** may also save, as part of the snapshot, associated data of modules that are executed in conjunction with the virtual controller, even if such modules are hosted on other nodes and/or simulate other physical components. In another example, when taking and saving a snapshot of a virtual CIOC, the VMN **300** may save data corresponding to the virtual machine level of the virtual CIOC, and/or the VMN **300** may save process simulation values observed by the virtual CIOC.

(88) Dynamic Simulation

(89) As mentioned above, the MPDSC system **10** supports the dynamic simulation of the entire process plant **100** and/or of portions thereof. For example, the MPDSC system **10** provides for a real-time simulation, where the real-time simulation mirrors the run-time timing, values, and/or other behaviors of at least part of a corresponding physical component and/or physical operational process. As utilized herein, the terms “simulation” and “simulation run” are utilized interchangeably, and a simulation or simulation run is bounded. That is, each simulation or simulation run has a respective start and a respective finish, which may be defined as desired, e.g., based on a time, a parameter value, a state, a user command, and/or other criteria.

(90) Additionally, the MPDSC system **10** provides for manipulation of simulations such as, for example, speeding up or slowing down the pace of simulation execution, pausing the simulation, inserting and/or modifying various values, changing initial conditions, etc. Simulations may be executed entirely within the virtual environment **12** of the MPDSC system **10**, and/or simulations may be executed using one or more virtual or simulated components in conjunction with one or more physical components (e.g., a configuration of a controller simulated by a Virtual Node in the virtual environment **12** may communicate with a physical field device disposed in the physical environment **15** to test the simulated controller behavior). Generally speaking, the MPDSC system **10** provides a platform via which engineers and users may test and check out draft control strategies and draft operator displays, investigate process improvements, perform operator training, perform case studies, etc. The VMN **300** manages, coordinates, and drives simulation activities supported by the MPDSC system **10**.

(91) In particular, the VMN **300** creates, configures, and administrates virtual nodes **25**, **30x** and other virtual components **28**, **32**, **35**, **38**, **42x**, **60** that simulate at least portions of the physical plant **100**. Virtual nodes **25**, **30x**, that are utilized solely for simulation purposes (and not for run-time control purposes) are interchangeably referred to herein as “simulated nodes.” For example, the VMN **300** may generate a mirror simulation system of the entire run-time control system as described by the plant configuration database **172b**, e.g., by utilizing simulated nodes that simulate integral physical hardware components such as controllers **110**, workstations **170a**, and/or other physical devices and their corresponding network interconnections (in some implementations, simulating down to the MAC address level). In another example, the VMS **300** may generate respective simulations of one or more individual physical components, each of which may execute

in a stand-alone mode or in conjunction with one or more other simulated, virtual, and/or physical components. For instance, the VMS **300** may generate, within the virtual environment **12**, a virtual twin of a physical component that is currently operating within the physical environment **15**, e.g., to test a new control configuration, an upgrade, a patch, etc. that is to be applied to the physical component. In another example, VMS **300** may generate a simulation of a MAC address level behavior of a particular physical component. In yet another example, the VMS **300** may generate a unitary (e.g., single) simulated node that simulates the real-time operating behavior of a group of physical devices, components, or nodes, such as a control loop or an operator display view.

(92) With regard to configuring the types of data values utilized within the virtual **12** and physical **15** environments of the process control system **100**, in an embodiment, a system process data definition (which may be configured via a system configuration application **172a** and/or via the VMS **300**) defines which data values (and/or types thereof) utilized within the process plant **100** are to be simulated, and the VMS **300** assigns respective Data Tags (e.g., identifiers) to the data values (and/or types thereof) that are to be simulated. For example, the VMS **300** may automatically assign Data Tags to at least some data values and/or data types that are to be simulated, and/or the VMS **300** may receive manually generated Data Tags of at least some data values and/or data types that are to be simulated, e.g., via the user interface **302**. Assigned Data Tags of simulated data values and/or data types may be stored in the virtual environment configuration database **305** and/or in the system configuration database **172b**.

(93) Additionally, the VMN **300** drives and/or coordinates the simulations of the entire physical plant **100** and/or portions thereof. To this end, the virtual environment **12** provides a Simulator API (Application Programming Interface) **310** or other suitable access mechanism via which applications, such as the user interface **302**, IT layer applications **21x**, and/or third-party or external applications **22x** may interface with the virtual environment **12** for the purposes of simulation. In an embodiment, the VMN **300** or other computing device within the virtual environment **12** hosts and/or exposes the Simulator API **310** to other applications.

(94) As shown in FIG. 5, the Simulator API **310** communicates simulated data values (also interchangeably referred to herein as “simulation data values,” “simulated values,” and/or “simulation values”) to and from the virtual environment **12** and simulated components included therein via the I/O Switch **25**. Generally speaking, simulated run-time process data is delivered between **28**, **30x**, and the Simulator API **310** via the nodes' respective Pub/Sub layers. For example, simulated nodes **28**, **30x**, **310** may publish run-time process data to one or more other simulated nodes **28**, **30x**, **310** via the I/O Switch **25**, and simulated nodes **28**, **30x**, **310** may subscribe to run-time process data that is generated by one or more other simulated nodes **28**, **30x**, **310** and received via the I/O switch **25**. On the other hand, in an embodiment, simulated data values that are not run-time process data (e.g., data values that are configurable and that are typically assigned during down-load time in the physical environment **15**) may be delivered between simulated nodes **28**, **30x**, **310** on an as-changed basis, e.g., only when respective values of the data change.

(95) The Simulator API **310** is configured with names, data tags, identifiers, data definitions, channel data, etc. of simulated components within the virtual environment **12** (e.g., based on data stored in the virtual configuration database **305**), and the Simulator API **310** is also configured to communicate using one or more industrial and/or general purpose communication/data protocols, which may be a standardized protocol, e.g., OPC, Ethernet, IPv6, etc. As such, the Simulator API **310** serves as a data delivery and transport mechanism between simulated components within the virtual environment **12** and other applications **302**, **21x**, **22x** that are consumers and/or providers of simulation data and that utilize the one or more industrial and/or general purpose communication/data protocols. For example, a third-party simulation application may utilize the Simulator API **310** to inject test values for use by a simulated component, change the initial conditions of a simulation run, etc.

(96) Additionally or alternatively, the Simulator API **310** handles simulation commands provided

via the user interface **302** and/or by other applications **21x**, **22x**. For example, the Simulator API **310** may receive simulation commands to speed up and/or slow down the pace of module execution (at least some of which is simulated within the virtual environment **12**) and/or to speed up and/or slow down the pace of I/O processing, and may drive the timing and the actions of associated simulated nodes **30x**, **28** and/or of the virtual communication network **45** accordingly. In another example, the Simulator API **310** may receive simulation commands to inject and/or change various data values during the simulation. Further, the Simulator API **310** may receive administrative simulation commands to, for example, save a snapshot, retrieve a saved snapshot, restore from saved data, etc. and may perform the corresponding action in response. For example, the Simulator API **310** may receive and respond to simulation commands to save, retrieve, restore, etc. a simulation of the entire process plant **100** (e.g., to/from the virtual environment configuration database **305** and/or from to/from the system configuration database **172b**) as well as to save, retrieve, restore, etc. a simulation of one or more nodes or components of the process plant **100**.

(97) Generally speaking, the Simulation API **310** is stateful. That is, the Simulation API **310** is aware of various states of the simulation and components (e.g., of virtual, physical, and/or simulated devices; virtual, physical, and/or simulated modules; other types of virtual, physical, and/or simulated components; a process that is being at least partially simulated by the simulation; a state of the overall simulation; etc.) associated therewith as the simulation executes, and responds to the received commands based on the various states. For example, in responding to a Save command, the Simulation API **310** may save process data in conjunction with data indicative of associated components' states. In another example, when simulating a modified control module executing within a controller for testing purposes, the Simulation API **310** may allow a user to change the state of an associated module to determine how the modified control module would respond to different states of the associated module.

(98) FIG. **6** is a flow diagram of an example method **400** of controlling a node of a process control system of an industrial process plant. At a least a portion of embodiments of the method **400** may be performed by one or more portions of the multi-purpose dynamic simulation and run-time industrial or process control (MPDSC) system or platform **10** of FIG. **1** and/or any one or more components thereof; by one or more portions of embodiments of the virtual nodes **52a**, **52b** of FIG. **2**; by one or more portions of embodiments of the physical plant environment **100** of FIG. **3** and/or any one or more components thereof; by one or more portions of embodiments of the arrangement **200** of I/O Switches of FIG. **4**; and/or by one or more portions of embodiments of the Virtualization Management Node **300** of FIG. **5**. For ease of illustration, and not for limitation purposes, the method **400** is described herein with simultaneous reference to portions of FIGS. **1-5**. Further, the method **400** may include more, fewer, and/or alternate steps other than those described herein, in embodiments.

(99) As illustrated in FIG. **6**, the method **400** includes communicatively connecting (block **402**) a field device and a virtual node via an I/O switch disposed between the field device and the virtual node so that the field device, the I/O switch, and the virtual node operate in conjunction during run-time operations of the industrial process plant, e.g., to control an industrial process. For example, the field device may be similar to one of the field devices **125-132** and **140-146** shown in FIG. **3**, one of the physical nodes **40a**, **40b**, **40c**, or PNa-PNk of FIG. **1**, and/or one of the physical nodes **212a-212n**, **215a-215n**, **218a-218n**, or **220a-220n** shown in FIG. **4**. The I/O switch may be implemented as an embodiment of the I/O switch **25** of FIG. **1** and/or one of the I/O switches **202-210** of FIG. **4**, for example. The virtual node may be similar to one of the virtual nodes **30a**, **30b**, **30c**, . . . , **30p** shown in FIG. **1**, the virtual nodes **52a**, **52b** shown in FIG. **2**, and/or the virtual nodes **212a-212n**, **215a-215n**, **218a-218n**, or **220a-220n** shown in FIG. **4**, for example.

(100) At any rate, with respect to the block **402** of the method **400**, the field device that is communicatively connected to the virtual node via the I/O switch is disposed in a physical environment of the industrial process plant and performs a physical function, during run-time

operations of the industrial process plant, to control an industrial process to generate physical products from raw or other types of input materials, such as manufacturing, material transformation, etc. For example, the field device may itself be a valve or another type of device, may actuate another device such as a valve or other type of device, may increase or decrease a temperature, may take a measurement, may sense a condition, etc.

(101) With respect to FIG. 6, the I/O switch is a subscriber to first data that is generated by the field device and that has been published, and the I/O switch is a publisher of second data that is indicative of the first data generated by the field device. For example, the I/O switch may receive, via a subscription, first data that is generated by the field device, e.g., while the field device is operating to control the industrial process during run-time operations of the plant, and the I/O switch may publish the received first data as second data. Generally, the I/O switch may publish and subscribe to various data of various devices to which it is communicatively connected via one or more publish/subscribe layers or mechanisms, such as the pub/sub layers 32a-32p, 35, 38, and 42a-42c shown in FIGS. 1 and 5, and/or the pub/sub layers 55a, 55b of FIG. 2.

(102) Additionally with respect to FIG. 6, the virtual node is disposed in a virtual environment of the industrial process plant, and is a subscriber to the second data corresponding to the field device and published by the I/O switch. For example, the virtual node may subscribe to various data of various devices to which it is communicatively connected, and may receive the various data to which the virtual node has subscribed (or indications thereof) via the virtual node's publish/subscribe layer or mechanism, such as one of the pub/sub layers 32a-32p of FIG. 1 or one of the pub/sub layers 55a, 55b of FIG. 2. Similarly, the virtual node may publish data for other nodes' consumption via its respective pub/sub layer.

(103) In some arrangements, the virtual node may be a virtual run-time node, such as a virtual controller, virtual safety controller or logic solver, I/O card or marshalling system, or other type of node which is a virtualized instance of a physical node which may be utilized in the physical environment of the industrial plant, e.g., such as described above. In some implementations, a single virtual run-time node may virtualize a group of nodes. For example, a single virtual run-time node may behave as a physical I/O marshalling cabinet and the set of physical I/O cards included therein.

(104) In some arrangements, the virtual node may be a simulated node which simulates one or more integral hardware physical components of the physical environment of the process plant and optionally behaviors and/or operations thereof, which may be, for example, sub-systems, devices, components of devices, networking connections and/or components, etc., down to the granularity of MAC address levels, if desired.

(105) At any rate, the virtual node (whether the virtual node is a virtual run-time node or is a simulated node) may include a respective component behavior module (CBM), which may comprise a set of instructions stored on one or more memories of the virtualized node and executed by one or more processors of the virtualized node. The CBM that is resident on and executed at the virtualized node may govern at least some behaviors and/or operations of the virtualized node, e.g., in a manner such as described above, at least by operating on I/O data. For example, the CBM of the virtualized node may be implemented as a control module or routine, a safety logic module or routine, an executable object, etc. Generally speaking, an instance of the CBM of the virtualized node is able to be utilized by a physical node of the process plant. That is, different instances of a particular CBM may be downloaded or otherwise provided to a virtual node and to a physical node, and the downloaded instances of the particular CBM may be unaware or agnostic as to whether they are being executed in a virtual node or in a physical node. With particular respect to the virtual node of the method 400, the virtual node publishes data generated by its CBM via its pub/sub layer, and the virtual node receives published data to which it has subscribed via its pub/sub layer and provides the received data to its CBM so that the CBM may operate on the received data.

(106) As shown in FIG. 6, during the run-time operations of the industrial process plant, the

method **400** includes receiving, by the virtual node, the second data corresponding to the field device that has been published by the I/O switch (block **405**). For example, the virtual node may receive, via its pub/sub layer, the second data published by the I/O switch, where the published second data is based on the first data that is generated by the field device and that is subscribed to and received by the I/O switch.

(107) At a block **408**, the method **400** includes operating on, during the run-time operations of the industrial process plant and by utilizing a component behavior module of the virtual node, the second data, thereby generating a control signal based on the second data. For example, the CBM that is resident on and executed at the virtual node may operate on the received second data (e.g., may utilize the received second data as input(s)), and may generate a corresponding control signal based on the operations or execution of the CBM. The CBM of the virtual node may be a control routine or module; a safety logic routine, module, or solver; or another type of object, routine, module, or executable, for instance.

(108) At a block **410**, the method **400** includes causing, during the run-time operations of the industrial process plant, an indication of the control signal to be transmitted to another node of the process control system, e.g., a recipient node, thereby modifying a behavior of the another or recipient node (block **410**). The another or recipient node may be another virtual node (such as another virtual run-time node, or a simulated node), or the recipient node may be a physical node of the industrial process plant.

(109) In an embodiment of the block **410**, causing the indication of the control signal to be transmitted to the recipient node includes directly transmitting, by the virtual node, the indication of the control signal to the recipient node.

(110) In another embodiment of the block **410**, causing the indication of the control signal to be transmitted to the recipient node includes publishing, by the virtual node, third data that is indicative of the control signal, e.g., via the pub/sub layer of the virtual node. In this embodiment, the I/O switch may be a subscriber to the third data published by the virtual node, and the I/O switch may operate on the third data to generate and publish fourth data that is indicative of the received third data, e.g., via the pub/sub layer of the I/O switch. In an example implementation of this embodiment, the recipient node may be a subscriber to the fourth data published by the I/O switch, and as such the recipient node may receive the published fourth data via its pub/sub layer, where the fourth data is indicative of the control signal.

(111) In another example implementation of this embodiment, an intervening node may be communicatively disposed between the I/O switch and the recipient node. In this example implementation, the intervening node may be a subscriber to the fourth data published by the I/O switch, and as such the intervening node may receive the published fourth data via its pub/sub layer. The intervening node may convert or translate the received fourth data into a communication format that is native or otherwise known to the recipient node, and may transmit and/or publish the converted or translated fourth data via one or more suitable communication links and/or networks to the recipient node, so that the recipient node receives an indication of the control signal in its native or known communication format. For example, the intervening node may be similar to one of the nodes **40a-40c** or **28** shown in FIG. 1, or may be another I/O switch such as one of the I/O switches **202-210** of FIG. 4. In an example arrangement, the intervening node may be an I/O hub that is communicatively disposed between the I/O switch and a plurality of physical devices that are disposed in the physical environment of the process plant. The plurality of physical devices may or may not include the field device which generated the first data, e.g., as described above with respect to block **402**. In this example arrangement, the I/O hub may be a publisher of data generated by any of the plurality of physical devices to which it is communicatively connected, and may be a subscriber to data that is utilized by any of the plurality of physical devices and that is published by another virtual device, the I/O switch, or any other publishing node utilizing the MPDSC **10**.

(112) With further respect to the block **410**, whether the recipient node receives the fourth data published by the I/O switch directly via a subscription or via an intervening node, the content of the fourth data (which, as previously described, is indicative of the control signal generated by the CBM of the virtual node) received by the recipient node may cause the recipient node to modify its behavior. As previously discussed, the recipient node may be another virtual node (which may be a virtual run-time node or a simulated node) of the industrial process plant, or the recipient node may be a physical node of the industrial process plant. As such, the content of the fourth data that is received by the recipient node (via the I/O switch or via the intervening node) may be operated on by the CBM of the recipient node, thereby causing a change in the behavior and/or the operations, e.g., the run-time behavior and/or the run-time operations, of the recipient node. For example, the content of the received fourth data may be indicative of a control signal generated by the virtual node, and the CBM of the recipient node may operate on the control signal and modify its behavior and/or operations accordingly, e.g., while the industrial process plant is operating in run-time to control the industrial process. In some instances, depending on the role of the recipient node, at least a portion of the control of the industrial process within the process plant may be modified based on the content of the data received by the recipient node and corresponding to the fourth data published by the I/O switch.

(113) In an embodiment of the method **400** (not shown in FIG. 6), the virtual node described with respect to the block **402** is a virtual run-time node, and the method **400** includes communicatively connecting one or more simulated nodes to the I/O switch, where the one or more simulated nodes are virtual nodes disposed in the virtual environment of the industrial process plant. The one or more simulation nodes and at least a portion of the I/O switch may be included in a simulation system, for example, where the simulation system simulates the run-time behavior and/or run-time operations of one or more virtual and/or physical components and/or nodes of the industrial process plant. Indeed, in some scenarios, the one or more simulated nodes may simulate the run-time behavior of respective virtual and/or physical components and/or nodes in conjunction with the run-time operations of one or more virtual run-time and/or physical nodes of the industrial process plant, e.g., while the one or more actual virtual run-time and/or physical nodes are operating during run-time while the industrial process plant is controlling the process.

(114) To provide the simulation of run-time behavior, each of the one or more simulation nodes may execute, via one or more processors, a respective CBM which is stored on one or more memories of the respective simulation node. As mentioned above, the respective CBMs of the simulated nodes are agnostic or unaware of whether they are executing on a simulated node, on a virtual run-time node, or on a physical node, and thus instances of the respective CBMs are deployable into various respective physical and/or virtual run-time nodes, components, and/or devices within the physical environment of the process plant. Generally speaking, a simulated node may simulate any physical device, or component thereof, that is deployable and/or disposed in the industrial process plant, including, but not limited to, for example: a process controller; a safety controller; a safety logic solver; an I/O node, card, or device; a wireless device; a Ethernet device; an operator workstation; a user interface device; a tool; a gateway; an electronic marshalling cabinet; a network connection; a MAC address; another type of physical device or component that is deployable and/or disposed within the physical environment of the industrial process plant; a module, a routine, a function or behavior; a MAC address; a hardware sub-component of a particular physical device or component; or another portion of a particular physical device or component that is deployable and/or disposed within the physical environment of the industrial process plant.

(115) In some embodiments, the method **400** includes executing a simulation run including a particular simulated node of the one or more simulated nodes of the system or MPDSC **10** (not shown in FIG. 6). In particular, executing the simulation run includes executing the respective CBM of the particular simulated node. The respective component behavior module of the particular

simulated node may be executed to thereby generate data; and the data generated by the particular simulated node may be communicated to another node of the MPDSC **10**, e.g., a receiving node of the data generated by the particular simulated node. For example, the data generated by the particular simulated node may be directly transmitted from the particular simulated node to the receiving node. In another example, the data generated by the particular simulated node may be published, via the pub/sub layer of the particular simulated node, and the receiving node, the I/O switch, or an intervening node communicatively disposed between the I/O switch and the receiving node may be a subscriber to the generated data published by the simulated node. The receiving node may receive the content of the generated data via the subscriber to the generated data published by the simulated node, such as in a manner described above. The receiving node may be another virtual node (e.g., another simulated node, or another virtual run-time node), or the receiving node may be a physical node. In some implementations, at least one of the simulated node or the receiving node may operate in conjunction with other run-time virtual and/or physical nodes during run-time operations of the industrial process plant.

(116) In some scenarios, subsequent to executing the simulation run that involves the particular simulation node, the method **400** may include downloading or otherwise providing a respective instance of the CBM of the particular simulation node to a virtual run-time node or to a physical node of the industrial process plant. For example, the simulation run may be a test or otherwise prove the desired behavior of the CBM in the particular simulated node, and after the simulated behavior has been proven to be satisfactory and/or has been approved, the instance of the CBM may be downloaded into a run-time node (whether virtual or physical). As such, via the MPDSC **10**, simulated behavior may be easily tested and proven (in some cases, under run-time conditions), and once approved, may be easily downloaded or provided for execution in respective virtual and/or physical run-time nodes.

(117) Not all simulation runs, though, need be executed in real-time or under run-time conditions. To this end, the simulation system may provide a simulation management node and a simulator access mechanism via which simulation runs may be administered, as desired, under various conditions. For example, the simulation management node may be implemented using an embodiment of the VMN **300** of FIG. 5, and the simulator access mechanism may be implemented by using an Application Programming Interface (API), a function, a service, and/or another type of mechanism that is accessible and exposed to users and/or other applications, such as the Simulator API **310** of FIG. 5. The method **400** may include utilizing the simulator access mechanism to, for example, provide and/or receive one or more simulated values utilized during the execution of the simulation run including the particular simulated node. Additionally or alternatively, the method **400** may include utilizing the simulator access mechanism to operate on one or more simulation execution and/or administrative commands, e.g., as provided by a user and/or by other applications (such as OT layer applications associated with the enterprise associated with or providing the industrial process control system and/or plant, and/or third-party applications). For example, the method **400** may include utilizing the simulator access mechanism to execute the simulation run at a run-time pace, speed up the simulation run to execute faster than the run-time pace; slow down the simulation run to execute slower than the run-time pace; set a simulated value of the simulation run; set an initial condition of the simulation run; pause the simulation run; set an intermediate condition of the simulation run; modify a pace of the execution of the simulation; modify a simulated value associated with the simulation run; save or store data associated with at least a portion of the simulation run; retrieve a saved or stored at least a portion of the simulation run; save or store a configuration of the simulation run; retrieve a saved or stored configuration of the simulation run; and/or the like, such as in a manner described above.

(118) In an embodiment, the simulator access mechanism may maintain one or more states associated with a simulation run. The one or more states may be respectively associated with various one or more portions of the simulation run and/or with components and/or devices

associated therewith. For example, the one or more states may correspond to a state of a virtual component, a state of a physical component, a state of a simulated component, a state of a virtual device, a state of a physical device, a state of a simulated device, a state of a virtual module, a state of a physical module, a state of a simulated module, a state of a process that is being at least partially simulated by the simulation run, a state of the simulation run, and/or another type of state associated with the simulation run, such as in a manner described above.

(119) Thus, in view of the above, the novel Multi-Purpose platform for Dynamic Simulation and run-time Control platform **10** provides numerous benefits and advantages over known process control systems. For example, as the MPDSC platform **10** supports both simulation and run-time control via virtual components, testing of changes (e.g., upgrades, patches, etc.) may be performed in the virtual environment **12** via a simulated component, and upon satisfactory checkout, the simulated component may be easily activated as a virtual component of the process plant **100** (e.g., “Load, Evaluate, Go,” bumpless or warm switchovers, etc.). As such, switchovers of simulated components to run-time components is more easily accomplished, and downtime due to upgrades, patches, and planned maintenance is reduced. Moreover, the amount of resources that are utilized to provide virtualized hot spares of various components and bring the virtualized hot spares on-line during failure or error scenarios is also greatly reduced.

(120) Further, the provision of run-time virtual components by the MPDS platform **10** allows for independence between hardware and software within the plant **100**. For example, controller software that is utilized during run-time of the process control system **100** may be upgraded in run-time-virtual controllers without needing to upgrade or change any physical controller hardware. Similarly, due to the hardware/software independence, the MPDS platform **10** allows for hardware of the process plant **100** to be upgraded independently of software upgrades.

(121) Still, the provision of run-time virtual components and the administration and/or management thereof (e.g., via the VMS **300** of the MPDSC platform **10**) also allows for easy and less costly system scalability, as additional virtual components may be easily implemented within the virtual environment **12** without needing to pay for, install, test, commission, and check out different physical hardware components and required cabinets, wiring, etc. Indeed, when additional virtual components are needed to support the system **100**, virtual components may be created as needed up to the processing and/or memory limits of the physical computing devices and/or hardware on which the virtual environment **12** is implemented, after which additional physical computing devices and/or hardware may simply be added thereto.

(122) The virtual environment **12** provided by the MPDS platform **10** enables the creation of various components' digital twins and/or virtual simulation of the entire process plant **100**, e.g., for testing, hot spares, upgrades, patches, planned maintenance, and/or other purposes. Digital twins (e.g., of particular components and of the entire process plant **100**) may be updated in lock-step with updates to their respective particular physical component(s). That is, state information may be passed, via the MPDSC platform **10**, from physical node(s) to virtual node(s). Indeed, the MPDSC platform **10** provides enhanced on-line testing of changes and/or different scenarios (e.g., “what-if” scenarios), and, in some situations, in conjunction with run-time virtual and/or physical components of the process plant **100**. Due to the common MPDS platform **10** (and in particular, the virtual environment **12** of the MPDS platform **10**) being architected and utilized for both simulation and run-time control purposes, off-simulation is easily accomplished without any configuration changes.

(123) Still further, as the MPDS platform **10** abstracts the I/O of the process plant **100** away from being specifically directly associated with particular hardware, I/O configuration within the plant **100** is more flexible and is easily changed and adapted not only during run-time, but also during upgrades, maintenance, failures, and the like. Significantly, communicative connections between I/O devices and other components (e.g., CIOCs and controllers) are no longer limited by physical ports that are available on physical I/O devices. As I/O is abstracted within the MPDS platform **10**,

any number of communicative connections between a virtual I/O device and other components are logically possible, up to the processing and/or memory limits of the physical computing devices and/or hardware on which the virtual environment **12** is implemented, after which additional physical computing devices and/or hardware may simply be added thereto. Similarly, due to the abstraction of I/O within the MPDS platform **10**, and control modules and/or other CBMs **58** may be assigned to any virtual host device or component without any concern for I/O location and/or loading of the host device or component, as is necessary when utilizing physical host devices or components. That is, as previously discussed, virtual components (whether utilized for simulation or for run-time control purposes) may operate utilizing I/O from physical components or from other virtual components. As such, the MPDS platform **10** is able to abstract (e.g., by utilizing the I/O Switch **25**) redundancy, retry, and other mechanisms that are presently tied to specific implementations within currently known techniques so that the abstractions may be utilized across in multiple, different types of applications, scenarios, and implementations, e.g., with selected customization and specialization of various abstractions for particular implementations and/or applications. Consequently, the MPDS platform **10** is able to support many more numbers and types of I/O systems (as compared to currently known techniques) in a consistent and easily scalable manner.

(124) When implemented in software, any of the applications, services, virtual devices, vertical machines, virtual entities, etc. described herein may be stored in any tangible, non-transitory computer readable memory such as on a magnetic disk, a laser disk, solid state memory device, molecular memory storage device, or other storage medium, in a RAM or ROM of a computer or processor, etc. Although the example systems disclosed herein are disclosed as including, among other components, software and/or firmware executed on hardware, it should be noted that such systems are merely illustrative and should not be considered as limiting. For example, it is contemplated that any or all of these hardware, software, and firmware components could be embodied exclusively in hardware, exclusively in software, or in any combination of hardware and software. Accordingly, while the example systems described herein are described as being implemented in software executed on a processor of one or more computer devices, persons of ordinary skill in the art will readily appreciate that the examples provided are not the only way to implement such systems.

(125) Thus, while the present invention has been described with reference to specific examples, which are intended to be illustrative only and not to be limiting of the invention, it will be apparent to those of ordinary skill in the art that changes, additions or deletions may be made to the disclosed embodiments without departing from the spirit and scope of the invention.

Claims

1. A system of an industrial process plant, the system comprising: a process control system including one or more control loops operating to control an industrial process during run-time operations of the industrial process plant, each control loop of the one or more control loops including a respective field device disposed in a physical environment of the industrial process plant and a respective process controller disposed in a virtual environment of the industrial process plant, and a first node of the each control loop publishing, via a virtual communication network of the process control system, data in a first publication to which an I/O switch subscribes, and the I/O switch publishing, via the virtual communication network, the data in a second publication to which a second node corresponding to the each control loop subscribes to thereby deliver respective abstracted I/O data between the respective field device and the respective process controller within the each control loop and thereby control the industrial process, the first node being one of the respective process controller or a third node corresponding to the respective field device, and the second node being the other one of the respective process controller or the third

- node, and the first node, the second node, the third node, and the I/O switch included in the virtual communication network.
2. The system of claim 1, wherein the process control system includes more than one control loop utilizing the I/O switch.
 3. The system of claim 1, wherein at least one of the one or more process controllers is a virtual process controller or a virtual safety controller utilizing the I/O switch.
 4. The system of claim 3, wherein at least one other one of the one or more process controllers is a simulated process controller utilizing the I/O switch.
 5. The system of claim 1, further comprising the I/O switch.
 6. The system of claim 1, wherein the I/O switch delivers abstracted I/O data of at least one control loop to more than one recipient node or device.
 7. The system of claim 1, wherein each of the I/O switch and the respective process controller includes a respective publish/subscribe layer via which the respective abstracted I/O data is delivered within the each process control loop.
 8. The system of claim 1, wherein: the respective process controller is included in a plurality of virtual nodes disposed in the virtual environment of the industrial process plant; and the I/O switch communicates at least a portion of the abstracted I/O data to at least one of: one or more other virtual nodes included in the plurality of virtual nodes, or one or more physical nodes of the industrial process plant.
 9. The system of claim 8, wherein at least one of: the plurality of virtual nodes include at least one of: a virtual process controller, a virtual safety controller; a virtual safety logic solver; a virtual I/O card, device, or node; a virtual wireless device; a virtual Ethernet device; a virtual operator workstation; a virtual user interface device; a virtual tool; a virtual gateway; a virtual electronic marshalling cabinet or system; a virtualization of another type of physical device or component disposed within a physical environment of the industrial process plant; or a simulated node simulating at least a portion of a physical node disposed in the physical environment of the industrial process plant; or the plurality of physical nodes includes one or more physical process controllers and/or one or more physical safety controllers.
 10. The system of claim 8, wherein the system further comprises a virtualization management node that creates, configures, and administrates the plurality of virtual nodes based on one or more configurations of the process control system stored in a configuration database of the process control system.
 11. The system of claim 10, wherein the virtualization management node further creates and configures the I/O switch.
 12. The system of claim 1, further comprising an edge gateway communicatively connected to one or more applications, each of the one or more applications being a respective consumer of a respective at least a portion of the abstracted I/O data.
 13. The system of claim 12, wherein each of the I/O switch, the respective process controller, and the edge gateway system includes a respective publish/subscribe layer via which one or more parts of the abstracted I/O data is sent and/or received.
 14. The system of claim 12, wherein the edge gateway system includes one or more security mechanisms utilized in delivering the respective at least the portions of the abstracted I/O data between the I/O switch and the one or more applications.
 15. A system of an industrial process plant, the system comprising: a process control system including one or more control loops operating to control an industrial process during run-time operations of the industrial process plant, each control loop of the one or more control loops including a respective field device disposed in a physical environment of the industrial process plant and a respective process control device disposed in a virtual environment of the industrial process plant, and the each control loop utilizing an I/O switch to deliver respective abstracted I/O data within the each control loop, wherein: at least one of the one or more respective process

control devices is a first virtual process controller or a first virtual safety controller utilizing the I/O switch, at least one other one of the one or more respective process control devices is a simulated process controller utilizing the I/O switch, the simulated process controller being a virtual run-time node operating as the simulated process controller during the run-time operations of the industrial process plant, and upon an approval of the simulated process controller, the virtual run-time node changes from operating as the simulated process controller to operating as a second virtual process controller or a second virtual safety controller during the run-time operations of the industrial process plant to control the industrial process.

16. The system of claim 1, wherein at least one of: the I/O switch is a subscriber to first data generated by the respective field device and a publisher of second data indicative of the first data, and the respective process controller is a subscriber to the second data and operates on contents of the second data to generate a control signal to control the industrial process; or the I/O switch is a subscriber to third data indicative of the control signal and published by the respective process controller and a publisher of fourth data indicative of the third data.

17. The system of claim 16, wherein at least one of: the respective field device or a first intervening device communicatively disposed between the I/O switch and the respective field device is a subscriber to the fourth data, and the field device operates responsively to contents of the fourth data; or another process control device or a second intervening device communicatively disposed between the I/O switch and the another process control device is a subscriber to the fourth data, and the another process controller operates on the contents of the fourth data to control the industrial process.

18. A system of an industrial process plant, the system comprising: a process control system including one or more control loops operating to control an industrial process during run-time operations of the industrial process plant, each control loop of the one or more control loops including a respective field device disposed in a physical environment of the industrial process plant and a respective process control device disposed in a virtual environment of the industrial process plant, and the each control loop utilizing an I/O switch to deliver respective abstracted I/O data within the each control loop, and wherein: (i) at least one of: the I/O switch is a subscriber to first data generated by the respective field device and a publisher of second data indicative of the first data, and the respective process control device is a subscriber to the second data and operates on contents of the second data to generate a control signal to control the industrial process; or the I/O switch is a subscriber to third data indicative of the control signal and published by the respective process control device and a publisher of fourth data indicative of the third data; (ii) at least one of: the respective field device or a first intervening device communicatively disposed between the I/O switch and the respective field device is a subscriber to the fourth data, and the field device operates responsively to contents of the fourth data; or another process control device or a second intervening device communicatively disposed between the I/O switch and the another process control device is a subscriber to the fourth data, and the another process controller operates on the contents of the fourth data to control the industrial process; and (iii) at least one of the first intervening device or the second intervening device is a respective I/O hub communicatively disposed between the I/O switch and a respective plurality of physical devices of the industrial process plant.

19. The system of claim 17, wherein the another process control device is a physical process controller or a physical safety controller.

20. A system of an industrial process plant, the system comprising: a process control system including one or more control loops operating to control an industrial process during run-time operations of the industrial process plant, each control loop of the one or more control loops including a respective field device disposed in a physical environment of the industrial process plant and a respective process control device disposed in a virtual environment of the industrial process plant, the one or more process control devices included in a plurality of virtual nodes

disposed in the virtual environment of the industrial process plant; and the each control loop utilizing an I/O switch to deliver respective abstracted I/O data within the each control loop, and the I/O switch communicates at least a portion of the abstracted I/O data to at least one of: one or more other virtual nodes included in the plurality of virtual nodes, or one or more physical nodes of the industrial process plant; and a virtualization management node that creates, configures, and administrates the plurality of virtual nodes based on one or more configurations of the process control system stored in a configuration database of the process control system, and administrates the plurality of virtual nodes responsive to one or more conditions of a physical computing platform that supports the virtual environment, the physical computing platform including one or more computing devices, and the one or more conditions of the physical computing platform dynamically occurring during run-time operations of the industrial process plant; and the administration of the plurality of virtual nodes by the virtualization management node responsive to the dynamic occurrence of the one or more conditions of the physical computing platform includes at least one of: a deletion of a virtual node included in the plurality of virtual nodes, a creation of another virtual node, an activation of a stand-by virtual node, a re-allocation of resources, storing a respective configuration of at least a portion of the virtual node, storing run-time data corresponding to the at least the portion of the virtual node, storing simulation data corresponding to the at least the portion of the virtual node, storing a state of the at least the portion of the virtual node, storing other data corresponding to the at least the portion of the virtual node, restoring the at least the portion of the virtual node from respective stored data, or another administrative action.
